بسم الله الرّحمن الرّحیم

Scientific-Analytical Teyf Group

Practical Linear Algebra for Data Science and Machine Learning

Seyyed Hossein Hosseini

November 2024

Chapter 5:

Practical Linear Algebra for Data Science and Machine Learning

Outline $\left\{ \begin{array}{c} Vectors \\ Matrices \\ Matrix\ Decompositions \\ Tensors \end{array} \right.$

- If you want to work in any computational or technical field, you need to understand linear algebra.
- Linear algebra is the mathematical basis of nearly all algorithms and analyses implemented in computers.
- But the way it's presented in decades-old textbooks is much different from the way professionals use linear algebra today to solve real-world problems.
- Many operations and applications in linear algebra are actually quite simple and sensible, but they require a significant amount of background knowledge to understand. That's a good thing: the more linear algebra you know, the easier linear algebra becomes.

Practical Linear Algebra for Data Science, Mike Cohen

Mathematical Proofs Versus Intuition from Coding

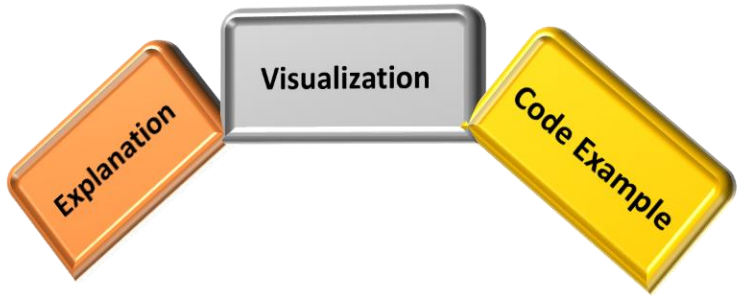
- **Question:** How do you understand math? Let us count the ways:

  1. *Rigorous proofs*
     A **proof** in mathematics is a sequence of statements showing that a set of assumptions leads to a logical conclusion. Proofs are unquestionably important in pure mathematics.

  2. *Visualizations and examples*
     Clearly written explanations, diagrams, and numerical examples help you **gain intuition** for concepts and operations in linear algebra. Most examples are done in **2D or 3D** for easy visualization, but the principles also apply to **higher dimensions.**

- The difference between these is that formal mathematical proofs provide rigor but rarely intuition, whereas visualizations and examples provide lasting intuition through hands-on experience but can risk inaccuracies based on specific examples that do not generalize.

- For proofs, you can see other courses or books, but we focus more on **building intuition** through:



- **Definition**

> **Linear algebra** is a branch of mathematics that deals with **vectors** and **matrices** and, more generally, with **vector spaces** and **linear transformations**.

- Linear algebra is widely used throughout science and engineering for **modeling** natural phenomena. It allows for efficient computation with models **representing** these phenomena.
- A good understanding of linear algebra is essential for understanding and working with many machine learning algorithms.
- For instance, **datasets** can be **represented** as **matrices**, where each **feature** and **sample** is expressed as a **vector** in the matrix.
- **Definitions of several types of mathematical objects**

> **Scalar:** A scalar is just a single number, In contrast to most of the other objects studied in linear algebra, which are usually arrays of multiple numbers.

- We usually give scalars **lower-case** variable names.
- When we introduce them, we specify what kind of number they are. For example, we might say "Let $s \in \mathbb{R}$ be the slope of the line," while defining a real-valued scalar, or "Let $n \in \mathbb{N}$ be the number of units," while defining a natural number scalar.

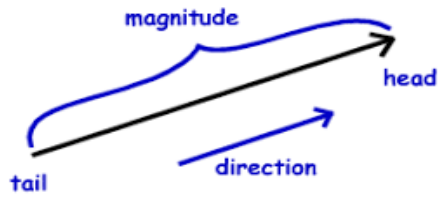
Vector

- **Definition**

> **Vector:** A vector is an **array** of numbers. The numbers are arranged in order. We can identify each individual number by its index in that ordering.

- Typically we give vectors **lower case** names written in **bold typeface**, such as $\boldsymbol{x}$.
- The elements of the vector are identified by writing its name in italic typeface, with a subscript. The first element of $\boldsymbol{x}$ is $\boldsymbol{x}_1$, the second element is $\boldsymbol{x}_2$ and so on.
- We also need to say what kind of numbers are stored in the vector. If each element is in $\mathbb{R}$, and the vector has n elements, denoted as $\mathbb{R}^n$.
- When we need to **explicitly** identify the elements of a vector, we write them as a column enclosed in square brackets:
- We can interpret/think of vectors as **identifying points in space**, with each element giving the coordinate along a different axis.

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$



Vector

A simple visual representation of a 2-dimensional vector space and two vectors within it

Properties of Vector Operations

Let $\boldsymbol{u}$, $\boldsymbol{v}$, and $\boldsymbol{w}$ be vectors, and let a and b be scalars.

Property	Statement
Associativity of vector addition	$\boldsymbol{u} + (\boldsymbol{v} + \boldsymbol{w}) = (\boldsymbol{u} + \boldsymbol{v}) + \boldsymbol{w}$
Commutativity of vector addition	$\boldsymbol{u} + \boldsymbol{v} = \boldsymbol{v} + \boldsymbol{u}$
Identity element of vector addition	There exists an element $\mathbf{0} \in V$, called the zero vector, such that $\boldsymbol{v} + \boldsymbol{0} = \boldsymbol{v}$ for all $\mathbf{v} \in V$.
Inverse elements of vector addition	For every $\boldsymbol{v} \in V$, there exists an element $-\boldsymbol{v} \in V$, called the additive inverse of $\mathbf{v}$, such that $\boldsymbol{v} + (-\boldsymbol{v}) = \boldsymbol{0}$.
Compatibility of scalar multiplication with field multiplication	$a(b\boldsymbol{v}) = (ab)\boldsymbol{v}$
Identity element of scalar multiplication	$1\boldsymbol{v} = \boldsymbol{v}$
Distributivity of scalar multiplication with respect to vector addition	$a(\boldsymbol{u} + \boldsymbol{v}) = a\boldsymbol{u} + a\boldsymbol{v}$
Distributivity of scalar multiplication with respect to field addition	$(a + b)\boldsymbol{v} = a\boldsymbol{v} + b\boldsymbol{v}$

- A vector of all zeros is called the **zeros vector**, indicated using a boldfaced zero, $\boldsymbol{0}$, and is a special vector in linear algebra.

Vector

- Linear algebra **convention** is to assume that vectors are in **column orientation** unless otherwise specified.

- Conceptualizing vectors either **geometrically** or **algebraically** facilitates **intuition** in different applications, but these are simply two sides of the same coin. For example, the geometric interpretation of a vector is useful in **physics** (e.g., representing physical forces), and the algebraic interpretation of a vector is useful in data science (e.g., storing sales data over time).

- Oftentimes, linear algebra concepts are learned **geometrically in 2D graphs**, and then are expanded to **higher dimensions** using algebra.

- **Operations on Vectors**

  - **Adding Two Vectors**
    - To add two vectors, simply add each **corresponding element**.
    - As you might have guessed, vector addition is defined only for two vectors that have the same dimensionality; it is not possible to add, e.g., a vector in $\mathbb{R}^3$ with a vector in $\mathbb{R}^5$.
    - **Vector subtraction** is also what you'd expect: subtract the two vectors **element-wise**.
    - Python is implementing an operation called **broadcasting**. You will learn more about broadcasting later in Python.

$$\begin{bmatrix}27\\28\\29\end{bmatrix} + \begin{bmatrix}11\\12\\31\end{bmatrix} = \begin{bmatrix}38\\40\\60\end{bmatrix}$$

$$\begin{bmatrix}27\\28\\29\end{bmatrix} - \begin{bmatrix}11\\12\\31\end{bmatrix} = \begin{bmatrix}16\\16\\-2\end{bmatrix}$$

$$\begin{bmatrix}1 & 3 & 4\end{bmatrix} + \begin{bmatrix}11\\12\\31\end{bmatrix} = ?$$

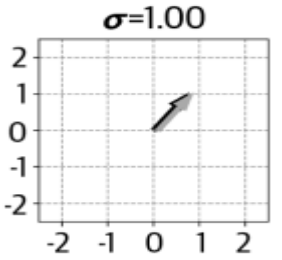
Vector

➢ Operations on Vectors

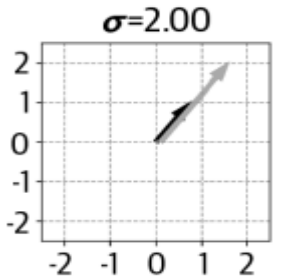
❑ Scalar-Vector Multiplication

- Vector-scalar multiplication is very simple: multiply each vector element by the scalar.

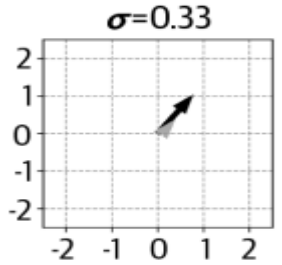
$$\sigma = 4,\ \boldsymbol{w} = \begin{bmatrix} 1 \\ 4 \\ 5 \end{bmatrix},\ \ \sigma\boldsymbol{w} = \begin{bmatrix} 4 \\ 16 \\ 20 \end{bmatrix}$$



- Why are scalars called "scalars"? That comes from the **geometric interpretation**. Scalars **scale** vectors.

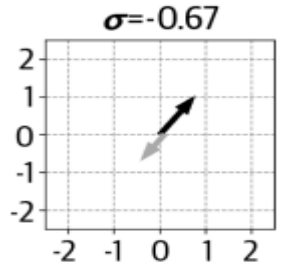


- **Vector Averaging:** Vector-scalar multiplication in combination with vector addition leads directly to **vector averaging**. Averaging vectors is the same as averaging numbers: sum and divide by the number of numbers. So, to average two vectors, add them and then scalar multiply by 0.5. In general, to average *N* vectors, sum them and scalar multiply the result by *1/N*



❑ Scalar-Vector Addition

- Adding a scalar to a vector is not formally defined in linear algebra: they are two separate kinds of mathematical objects and cannot be combined. However, numerical processing programs like Python will allow adding scalars to vectors, and the operation is comparable to scalar-vector multiplication: the scalar is added to each vector element.



Vector

➢ Operations on Vectors

❑ Transpose

- Transpose operation: it converts column vectors into row vectors, and vice versa.

- A slightly more formal definition that will generalize to transposing matrices: A matrix has rows and columns; therefore, each matrix element has a (*row, column*) index. The transpose operation simply swaps those indices.

$$\boldsymbol{m}_{i,j}^{T} = \boldsymbol{m}_{j,i}$$

- Vectors have either one row or one column, depending on their orientation. For example, a 6D row vector has $i = 1$ and j indices from 1 to 6, whereas a 6D column vector has i indices from 1 to 6 and $j = 1$. So swapping the i,j indices swaps the rows and columns.

- Here's an important rule: transposing twice returns the vector to its original orientation. In other words,

$$(\boldsymbol{b}^{T})^{T} = \boldsymbol{b}$$

- That may seem obvious and trivial, but it is the keystone of several important proofs in data science and machine learning.

Vector

➢ Operations on Vectors

❑ Vector Broadcasting in Python

- **Broadcasting** is an operation that exists only in **modern computer-based linear algebra**; this is not a procedure you would find in **a traditional linear algebra textbook**.

- Broadcasting essentially means to repeat an **operation** multiple times between one vector and each element of another vector.

- Because broadcasting allows for **efficient and compact computations**, it is used often in **numerical coding**.

$$\begin{bmatrix}11\\12\\31\end{bmatrix} + [1 \quad 3 \quad 4] = ?$$

➢ Vector Magnitude and Unit Vectors

- The **magnitude** of a vector—also called the **geometric length** or the **norm**—is the distance from tail to head of a vector, and is computed using the standard **Euclidean distance** formula: the square root of the sum of squared vector elements. Vector magnitude is indicated using double-vertical bars around the vector: $\| \boldsymbol{v} \|$.

$$\|\boldsymbol{v}\| = \sqrt{\sum_{i=1}^{n} v_i^2}$$

Vector

➢ Vector Magnitude and Unit Vectors

- In mathematics, the **dimensionality** of a vector is the number of elements in that vector, while the **length** is a **geometric distance**; in Python, the function **len()** (where len is short for *length*) returns the **dimensionality** of an array, while the function **np.norm()** returns the **geometric length** (**magnitude**).

- There are some applications where we want a vector that has a geometric length of one, which is called a ***unit vector*** **.**
- A unit vector is defined as $\| \boldsymbol{v} \| = 1$.

- **Creating a unit vector:**
  - Any **non-unit vector** has an associated **unit vector**. That means that we can create a unit vector in the same direction as a non-unit vector. Creating an associated unit vector is easy; you simply scalar multiply by the reciprocal of the vector norm:

$$\hat{\boldsymbol{v}} = \frac{1}{\|\boldsymbol{v}\|} \boldsymbol{v}$$

  - A unit vector (gray arrow) can be crafted from a non-unit vector (black arrow); both vectors have the same angle but different magnitudes.
  - Actually, the claim that "*any* non-unit vector has an associated unit vector" is not entirely true. There is a vector that has non-unit length and yet has no associated unit vector. Can you guess which vector it is?

Vector

➢ The Vector Dot Product

- The **dot product** (also sometimes called the **inner product**) is one of the most important operations in all of linear algebra. It is the **basic computational building block** from which many operations and algorithms are built, including **convolution**, **correlation**, the **Fourier transform**, **matrix multiplication**, **linear feature extraction, signal filtering**, and so on.

- There are several ways to indicate the dot product between two vectors. We will mostly use the common notation $\boldsymbol{a}^T\boldsymbol{b}$. In other contexts, you might see $\boldsymbol{a} \cdot \boldsymbol{b}$ or $\langle \boldsymbol{a}, \boldsymbol{b} \rangle$ **.**

- The dot product is **a single number** that provides information about the relationship between two vectors. Let's first focus on the algorithm to compute the dot product, and then I'll discuss how to **interpret** it.

- To compute the dot product, you multiply the corresponding elements of the two vectors, and then sum over all the individual products. In other words: element-wise multiplication and sum:

$$\delta = \sum_{i=1}^{n} a_i \, b_i$$

- **a** and **b** are vectors, and a_i indicates the *i*th element of **a**.

Vector

➢ The Vector Dot Product

- You can tell from the formula that the dot product is valid only between two vectors of the **same dimensionality**.
- A numerical example for dot product calculation:

$$\begin{aligned}[1\ 2\ 3\ 4]\cdot[5\ 6\ 7\ 8] &= 1\times5+2\times6+3\times7+4\times8\\ &= 5+12+21+32\\ &= 70\end{aligned}$$

- Now you know how to compute the dot product. What does the dot product **mean** and how do we **interpret** it?

- The dot product can be **interpreted** as a measure of **similarity** between two vectors. Imagine that you collected height and weight data from 20 people, and you stored those data in two vectors. You would certainly expect those variables to be related to each other (taller people tend to weigh more), and therefore you could expect the dot product between those two vectors to be large.

- On the other hand, the magnitude of the dot product depends on the **scale of the data**, which means the dot product between data measured in grams and centimeters would be larger than the dot product between data measured in pounds and feet. This arbitrary scaling, however, can be eliminated with a **normalization factor**.

- In fact, the **normalized dot product** between two variables is called the **Pearson correlation coefficient**, and it is one of the most important analyses in data science.

Vector

➢ The Vector Dot Product

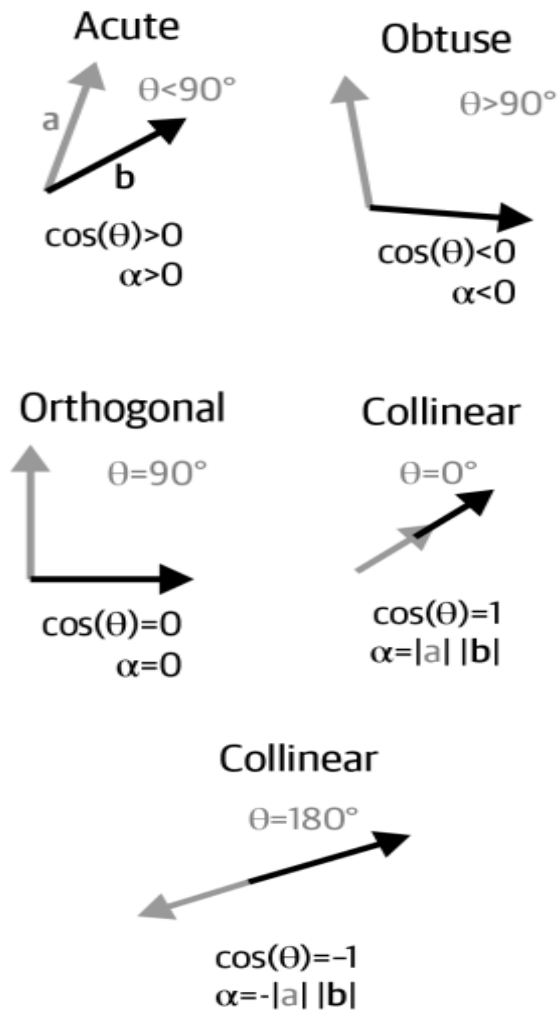
- **The Dot Product Is Distributive**
  - The distributive property of mathematics is that a(b + c) = ab + ac. Translated into vectors and the vector dot product, it means that:

$$\boldsymbol{a}^T(\boldsymbol{b}+\boldsymbol{c}) = \boldsymbol{a}^T\boldsymbol{b} + \boldsymbol{a}^T\boldsymbol{c}$$

- **Geometry of the Dot Product**
  - There is also a geometric definition of the dot product, which is the product of the magnitudes of the two vectors, scaled by the cosine of the angle between them

$$\delta = \|\boldsymbol{a}\|\,\|\boldsymbol{b}\|\,cos(\theta_{\boldsymbol{a},\boldsymbol{b}})$$

  - Notice that vector magnitudes are strictly positive quantities (except for the zeros vector, which has $\|\,\boldsymbol{0}\,\| = 0$), while the cosine of an angle can range between -1 and +1. This means that the sign of the dot product is determined entirely by the geometric relationship between the two vectors. Figure shows five cases of the dot product sign, depending on the angle between the two vectors (in 2D for visualization, but the principle holds for higher dimensions).



Vector

➢ Other Vector Multiplications

- The dot product is perhaps the most important, and most frequently used, way to multiply vectors. But there are several other ways to multiply vectors.

- **Hadamard Multiplication**
  - This is just a fancy term for element-wise multiplication. To implement Hadamard multiplication, each corresponding element in the two vectors is multiplied. The product is a vector of the same dimensionality as the two multiplicands. For example:

$$\begin{bmatrix}5\\4\\8\\2\end{bmatrix} \odot \begin{bmatrix}1\\0\\.5\\-1\end{bmatrix} = \begin{bmatrix}5\\0\\4\\-2\end{bmatrix}$$

  - **Example:** Hadamard multiplication is a convenient way to organize multiple scalar multiplications. For example, imagine you have an **IoT system** that monitors **energy consumption** in different **smart homes** and the **cost per unit of energy(**the cost per kilowatt-hour (kWh)**)** at each home. You could **represent** each variable as a vector, and then Hadamard-multiply those vectors to compute the **energy cost per home** (this is different from the total energy cost across all homes, which would be computed as the dot product).

Vector

➢ Other Vector Multiplications

▪ Outer Product

- The outer product is a way to create a matrix from a column vector and a row vector. Each row in the outer product matrix is the row vector scalar multiplied by the corresponding element in the column vector. We could also say that each column in the product matrix is the column vector scalar multiplied by the corresponding element in the row vector.

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} [d \;\; e] = \begin{bmatrix} ad & ae \\ bd & be \\ cd & ce \end{bmatrix}$$

- The outer product is quite different from the dot product: it produces a matrix instead of a scalar, and the two vectors in an outer product can have different dimensionalities, whereas the two vectors in a dot product must have the same dimensionality.

- The outer product is indicated as $\boldsymbol{v}\,\boldsymbol{w}^T$ (remember that we assume vectors are in column orientation; therefore, the outer product involves multiplying a column by a row). Note the subtle but important difference between notation for the dot product ($\boldsymbol{v}^T\boldsymbol{w}$) and the outer product ($\boldsymbol{v}\,\boldsymbol{w}^T$).

- The outer product is similar to broadcasting, but they are not the same: **broadcasting** is a general **coding operation** that is used to expand vectors in arithmetic operations such as addition, multiplication, and division; the **outer product** is a **specific mathematical procedure** for multiplying two vectors

Vector

➢ Vector Sets

- A collection of vectors is called a **set**. You can imagine putting a bunch of vectors into a bag to carry around. Vector sets are indicated using capital italics letters, like *S* or *V*. Mathematically, we can describe sets as the following:

$$V = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$$

- **Example 1:** Imagine, for example, an **IoT system** that monitors environmental conditions in one hundred **smart cities**. You could store the data from each city in a three-element vector, and create a **vector set** containing one hundred vectors. Each vector could represent:
  1. **Temperature**: The average temperature in the city.
  2. **Humidity**: The average humidity level in the city.
  3. **Air Quality Index (AQI)**: The average air quality index in the city.

- **Example 2:** Imagine, for example, an IoT system that collects data from **connected cars** in one hundred cities. You could store the data from each city in a three-element vector, and create a vector set containing one hundred vectors. Each vector could represent:
  1. **Number of Active Connected Cars**: The total number of connected cars actively transmitting data in the city.
  2. **Average Speed**: The average speed of the connected cars in the city.
  3. **Number of Traffic Incidents**: The number of traffic incidents reported by the connected cars in the city.

- This **structured approach** allows for efficient **storage**, **retrieval**, and **analysis** of connected car data across multiple cities, enabling better monitoring and management of urban traffic and transportation systems.

Vector

➢ Linear Weighted Combination

- Linear weighted combination simply means scalar-vector multiplication and addition: take some set of vectors, multiply each vector by a scalar(some variables contributing more than others), and add them to produce a single vector:

$$\mathbf{w} = \lambda_1 \mathbf{v}_1 + \lambda_2 \mathbf{v}_2 + \ldots + \lambda_n \mathbf{v}_n$$

- It is assumed that all vectors $\boldsymbol{v}_i$ have the same dimensionality; otherwise, the addition is invalid. The λ_i can be any real number, including zero.

- Technically, you could rewrite the equation for subtracting vectors, but because subtraction can be handled by setting a λ_i to be negative, it's easier to discuss linear weighted combinations in terms of summation.

- **Example:**

$$\lambda_1 = 1,\ \lambda_2 = 2,\ \lambda_3 = -3, \quad \mathbf{v_1} = \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix},\ \mathbf{v_2} = \begin{bmatrix} -4 \\ 0 \\ -4 \end{bmatrix},\ \mathbf{v_3} = \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix}$$

$$\mathbf{w} = \lambda_1 \mathbf{v_1} + \lambda_2 \mathbf{v_2} + \lambda_3 \mathbf{v_3} = \begin{bmatrix} -7 \\ -4 \\ -13 \end{bmatrix}$$

- The concept of a linear weighted combination is the mechanism of creating **vector subspaces** and **matrix spaces,** and is central to **linear independence**. Indeed, **linear weighted combination** and the **dot product** are two of the most important elementary building blocks from which many advanced linear algebra computations are built.

Vector

➢ Linear Independence

- A set of vectors is **linearly dependent** if at least one vector in the set can be expressed as a linear weighted combination of other vectors in that set.

- And thus, a set of vectors is **linearly independent** if no vector can be expressed as a linear weighted combination of other vectors in the set.

- **Question:** linearly independent or linearly dependent?

$$V = \left\{ \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \begin{bmatrix} 2 \\ 7 \end{bmatrix} \right\} \qquad S = \left\{ \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \begin{bmatrix} 2 \\ 6 \end{bmatrix} \right\} \qquad T = \left\{ \begin{bmatrix} 8 \\ -4 \\ 14 \\ 6 \end{bmatrix}, \begin{bmatrix} 4 \\ 6 \\ 0 \\ 3 \end{bmatrix}, \begin{bmatrix} 14 \\ 2 \\ 4 \\ 7 \end{bmatrix}, \begin{bmatrix} 13 \\ 2 \\ 9 \\ 8 \end{bmatrix} \right\}$$

- **Question**: so how do you determine linear independence in practice? The way to determine linear independence is to create a matrix from the vector set, compute the **rank of the matrix**, and compare the rank to the smaller of the number of rows or columns. That sentence may not make sense to you now, because you haven't yet learned about matrix rank.

- **Independent Sets:** Independence is a property of a **set of vectors**. That is, a set of vectors can be linearly independent or linearly dependent; independence is not a property of an individual vector within a set.

Vector

➢ Subspace and Span

- For some (finite) **set of vectors**, the infinite number of ways to linearly combine them—using the same vectors but different numerical values for the weights creates a **vector subspace**.

- The mechanism of combining all possible linear weighted combinations is called the **span** of the vector set.

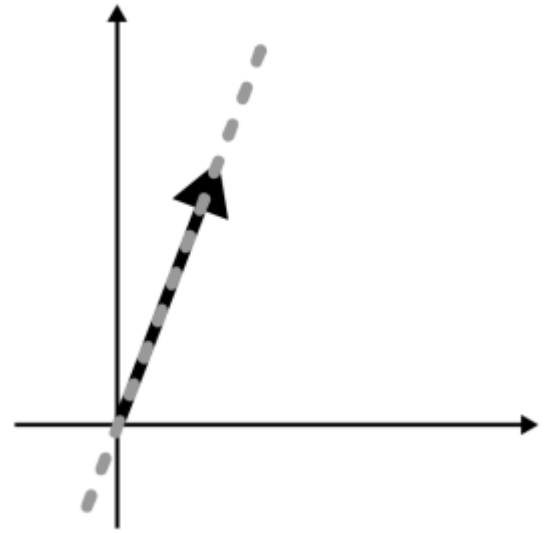
$$\text{span}(\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_N\}) \triangleq \left\{\sum_{i=1}^{N} \lambda_i \boldsymbol{v}_i : \lambda_i \in \mathbb{R}\right\}$$

- Let's work through a few examples. We'll start with a simple example of a vector set containing one vector:

$$V = \left\{\begin{bmatrix}1\\3\end{bmatrix}\right\}$$

- The span of this vector set is the infinity of vectors that can be created as linear combinations of the vectors in the set. For a set with one vector, that simply means all possible scaled versions of that vector.

- The figure shows the vector and the subspace it spans. Consider that any vector in the gray dashed line can be formed as some scaled version of the vector.



A vector (black) and the subspace it spans (gray)

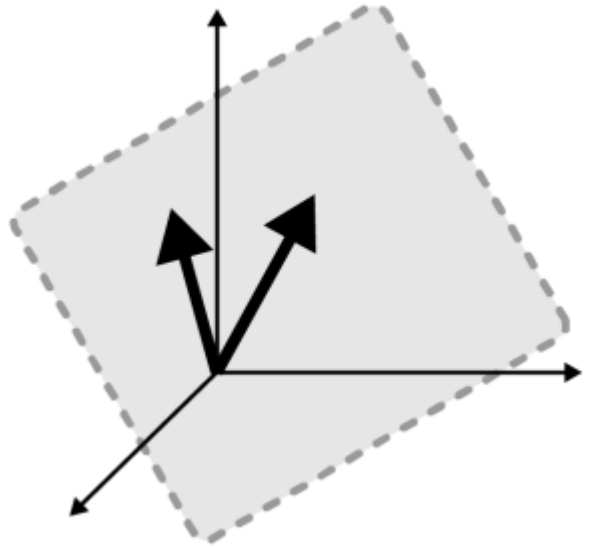
Vector

➢ Subspace and Span

- Our next example is a set of two vectors in $\mathbb{R}^3$:

$$V = \left\{ \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}, \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix} \right\}$$

- The vectors are in $\mathbb{R}^3$, so they are graphically represented in a 3D axis. But the subspace that they span is a 2D plane in that 3D space. That plane passes through the origin, because scaling both vectors by zero gives the zeros vector.



Two vectors (black) and the subspace they span (gray)

- The first example had one vector and its span was a 1D subspace, and the second example had two vectors and their span was a 2D subspace. There seems to be a pattern emerging—but looks can be deceiving. Consider the next example:

$$V = \left\{ \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} \right\}$$

Vector

➢ Subspace and Span

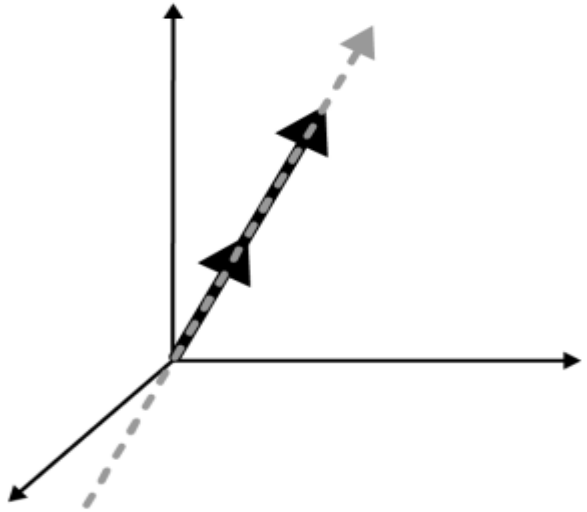
- Two vectors in $\mathbb{R}^3$, but the subspace that they span is still only a 1D subspace—a line. Why is that? It's because one vector in the set is already in the span of the other vector. Thus, in terms of span, one of the two vectors is redundant.

- **Question:** So then, what is the relationship between the dimensionality of the spanned subspace and the number of vectors in the set? You might have guessed that it has something to do with **linear independence**.

- The dimensionality of the subspace spanned by a set of vectors is the smallest number of vectors that forms a linearly independent set.

- If a vector set is linearly independent, then the dimensionality of the subspace spanned by the vectors in that set equals the number of vectors in that set.

- If the set is dependent, then the dimensionality of the subspace spanned by those vectors is necessarily less than the number of vectors in that set.



The 1D subspace (gray) spanned by two vectors (black)

Vector

➢ Subspace and Span

- Exactly how much smaller is another matter—to know the relationship between the number of vectors in a set and the dimensionality of their spanning subspace, you need to understand **matrix rank**, which you'll learn about in **Matrix Section.**

- The formal definition of a vector subspace is a subset that is closed under addition and scalar multiplication, and includes the origin of the space. That means that any linear weighted combination of vectors in the subspace must also be in the same subspace, including setting all weights to zero to produce the zeros vector at the origin of the space.

- Please don't lose sleep meditating on what it means to be "closed under addition and scalar multiplication"; just remember that a vector subspace is created from all possible linear combinations of a set of vectors.

- **Question:** What's the Difference Between Subspace and Span?
  - Span and subspace so often refer to identical mathematical objects that using the terms interchangeably is usually correct.
  - Thinking of **span** as a verb and **subspace** as a noun helps understand their distinction: a set of vectors spans, and the result of their spanning is a subspace. Now consider that a *sub*space can be a smaller portion of a larger space, as you saw in the figures. Putting these together: span is the mechanism of creating a subspace. (On the other hand, when you use span as a noun, then span and subspace refer to the same infinite vector set.)

Vector

➢ Basis

- A **basis** is like a ruler for measuring a space.
- **Example:** the distance that a **connected car** has traveled in the past 24 hours is the same, regardless of whether you measure it in meters, miles, or kilometers. But different units are more or less convenient for different problems.
- For instance, if you are monitoring the daily travel distance of connected cars in a city, using kilometers might be more practical for urban planning and traffic management. On the other hand, if you are analyzing the performance of a car's engine over short trips, meters might be more appropriate. Similarly, for long-distance travel analysis, miles could be more convenient.
- Back to linear algebra: a **basis** is a set of rulers that you use to describe the information in the matrix (e.g., data). Like with the previous examples, you can describe the same data using different rulers, but some rulers are more convenient than others for solving certain problems.
- The most common basis set is the **Cartesian axis**: the familiar XY plane that you've used since elementary school. We can write out the **basis sets** for the 2D and 3D Cartesian graphs as follows:

$$S_2 = \left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\} \qquad S_3 = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}$$

Vector

➢ Basis

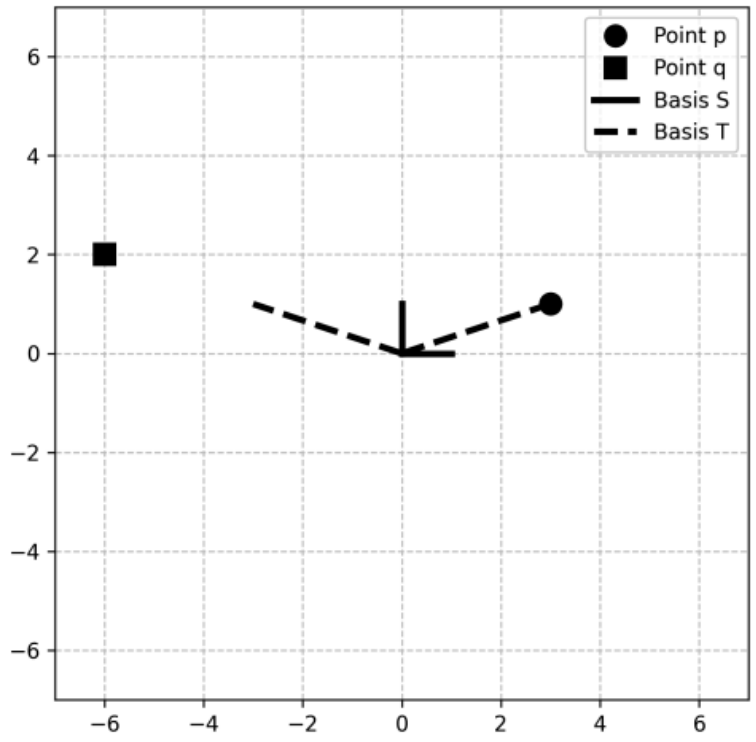
- Notice that the Cartesian basis sets comprise vectors that are mutually **orthogonal** and **unit length**. Those are great properties to have, and that's why the **Cartesian basis sets** are so ubiquitous (indeed, they are called the **standard basis set**).

- But those are not the only basis sets. The following set is a different basis set for $\mathbb{R}^2$.

$$T = \left\{ \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \begin{bmatrix} -3 \\ 1 \end{bmatrix} \right\}$$

- Basis set S_2 and T both span the same subspace (all of $\mathbb{R}^2$).

- **Question:** Why would you prefer T over S? Imagine we want to **describe** data points p and q in figure. We can describe those data points as their relationship to the origin that is, their coordinates—using basis S or basis T.



The same points (p and q) can be described by basis set S (black solid lines) or T (black dashed lines)

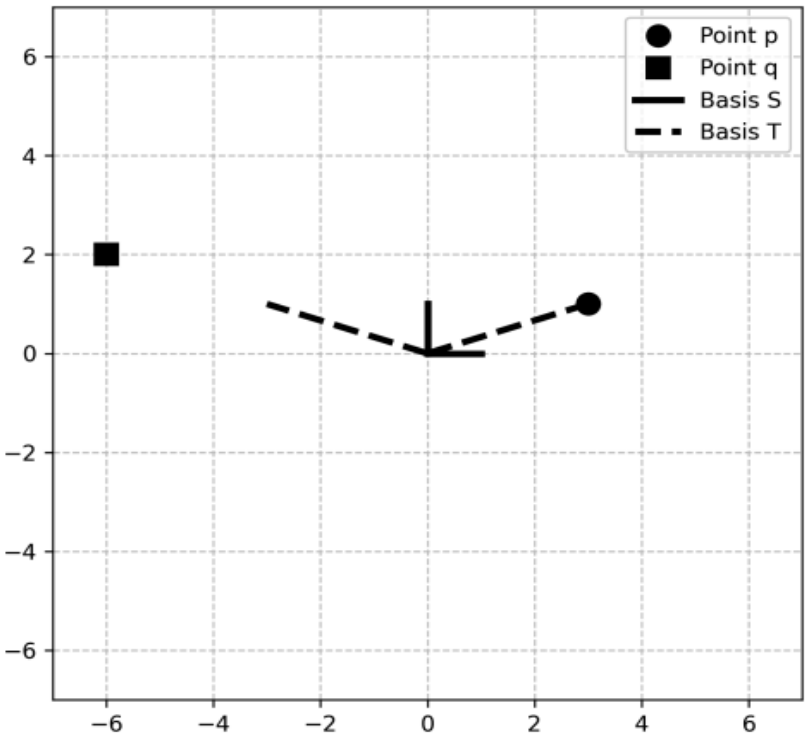
Vector

➢ Basis

- In basis *S*, those two **coordinates** are $p = (3, 1)$ and $q = (-6, 2)$. In linear algebra, we say that the points are expressed as the **linear combinations** of the **basis vectors**. In this case, that combination is $3\boldsymbol{s}_1 + 1\boldsymbol{s}_2$ for point *p*, and $-6\boldsymbol{s}_1 + 2\boldsymbol{s}_2$ for point *q*.

- Now let's **describe** those points in basis *T*. As coordinates, we have $p = (1, 0)$ and $q = (0, 2)$. And in terms of basis vectors, we have $1\boldsymbol{t}_1 + 0\boldsymbol{t}_2$ for point *p* and $0\boldsymbol{t}_1 + 2\boldsymbol{t}_2$ for point *q*.
- Again, the data points *p* and *q* are the same regardless of the basis set, but *T* provided a compact and orthogonal description.

- Bases are important in **data science** and **machine learning**. In fact, many problems in applied linear algebra can be conceptualized as finding the best set of basis vectors to describe some subspace. You've probably heard of the following terms: **dimension reduction**, **feature extraction**, **principal components analysis**, **independent components analysis**, **factor analysis**, **singular value decomposition**, **linear discriminant analysis**, **image approximation**, **data compression**. Believe it or not, all of those analyses are essentially ways of identifying optimal basis vectors for a specific problem.

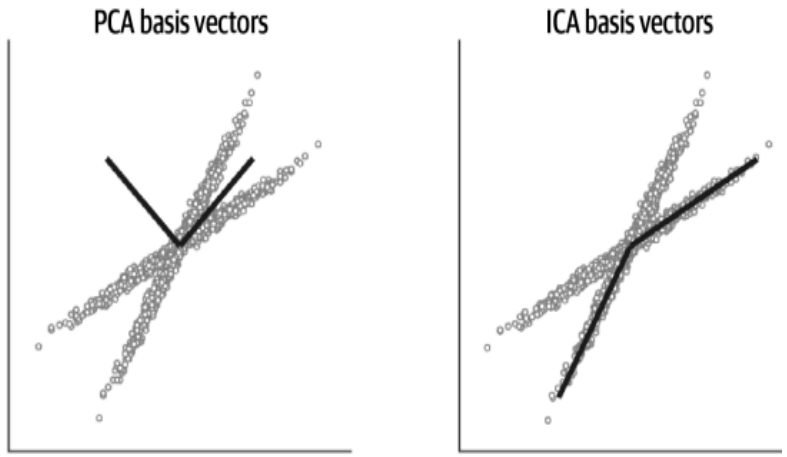


The same points (p and q) can be described by basis set S (black solid lines) or T (black dashed lines)

Vector

➢ Basis

- Consider figure: this is a dataset of two variables (each dot represents a data point). The figure actually shows three distinct bases: the "**standard basis set**" corresponding to the x = 0 and y = 0 lines, and basis sets defined via a **principal components analysis (PCA; left plot)** and via an **independent components analysis (ICA; right plot).**



A 2D dataset using different basis vectors (black lines)

- **Question:** Which of these basis sets provides the "best" way of describing the data? You might be tempted to say that the basis vectors computed from the ICA are the best. The truth is more complicated (as it tends to be): no basis set is intrinsically better or worse; different basis sets can be more or less helpful for specific problems based on the **goals of the analysis**, the **features of the data**, **constraints imposed by the analyses,** and so on.

Vector

➢ Basis

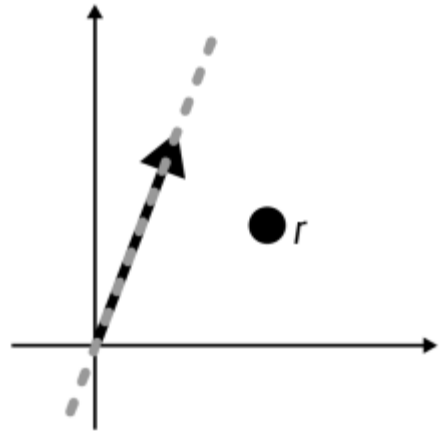
- Once you understand the concept of a basis and basis set, the formal definition is straightforward.

- **Definition**

> In fact, **basis** is simply the combination of span and independence: a set of vectors can be a basis for some subspace if it (1) spans that subspace and (2) is an independent set of vectors.

- The basis needs to span a subspace for it to be used as a basis for that subspace, because you cannot describe something that you cannot measure.

- Figure shows an example of a point outside of a 1D subspace. A basis vector for that subspace cannot measure the point ***r***. The black vector is still a valid basis vector for the subspace it spans, but it does not form a basis for any subspace beyond what it spans.



A basis set can measure only what is contained inside its span

- **Question**: So a basis needs to span the space that it is used for. That's clear. But why does a basis set require linear independence? The reason is that any given vector in the subspace must have a unique coordinate using that basis.

➢ Summary

- The beauty of linear algebra is that even the most sophisticated and computationally intense operations are made up of simple operations, most of which can be understood with **geometric intuition**.

- Do not underestimate the importance of studying simple operations on vectors, because what you learned in this vector section will form the basis for the rest of the course.

❖ Here are the most important take-home messages of this vector section:

- ❑ A vector is an ordered list of numbers that is placed in a column or in a row. The number of elements in a vector is called its **dimensionality**, and a vector can be represented as a line in a **geometric space** with the number of axes equal to the dimensionality.

- ❑ Several arithmetic operations (addition, subtraction, and Hadamard multiplication) on vectors work **element-wise.**

- ❑ The dot product is a single number that encodes the **relationship** between two vectors of the same dimensionality, and is computed as element-wise multiplication and sum.

- ❑ The dot product is zero for vectors that are **orthogonal**, which geometrically means that the vectors meet at a **right angle**.

- ❑ A vector set is a collection of vectors. There can be a finite or an infinite number of vectors in a set.

➢ Summary

- ❑ Linear weighted combination means to scalar multiply and add vectors in a set. Linear weighted combination is one of the single most important concepts in linear algebra.
- ❑ A set of vectors is linearly dependent if a vector in the set can be expressed as a linear weighted combination of other vectors in the set. And the set is linearly independent if there is no such linear weighted combination.
- ❑ A subspace is the **infinite set** of all possible linear weighted combinations of a set of vectors.
- ❑ A basis is a ruler for measuring a space. A vector set can be a basis for a subspace if it (1) spans that subspace and (2) is linearly independent.
- ❑ A major goal in data science and machine learning is to discover the best **basis set** to describe datasets or to solve problems.

Matrix

- **Definition**

> **Matrix:** A matrix is a 2-D array of numbers, so each element is identified by two indices instead of just one.

- We usually give matrices **upper-case** variable names with **bold typeface**, such as $\boldsymbol{A}$.
- If a real-valued matrix $\boldsymbol{A}$ has a height of m and a width of n, then we say that $\boldsymbol{A} \in \mathbb{R}^{m \times n}$.
- We usually identify the elements of a matrix using its name in italic but not bold font, and the indices are listed with separating commas. For example, $A_{1,1}$ is the upper left entry of $\boldsymbol{A}$.
- **Important reminder:** math uses 1-based indexing whereas **Python** uses 0-based indexing. Thus, element $A_{3,4}$ is indexed in Python as $A[2,3]$.
- We can identify all of the numbers with vertical coordinate i by writing a ":" for the horizontal coordinate. For example, $\boldsymbol{A}_{i,:}$ denotes the horizontal cross section of $\boldsymbol{A}$ with vertical coordinate i. This is known as the *i*-th *row* of $\boldsymbol{A}$. Likewise, $\boldsymbol{A}_{:,i}$ is the *i*-th *column* of $\boldsymbol{A}$.
- When we need to **explicitly** identify the elements of a matrix, we write them as an array enclosed in square brackets:

$$\boldsymbol{A} = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}$$

Matrix

- Matrices are highly versatile **mathematical objects**. They can **store** sets of equations, geometric transformations, the positions of particles over time, financial records, and myriad other things.

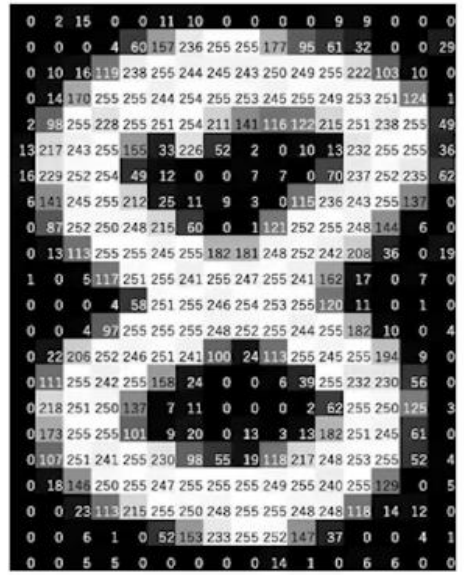
- In data science, matrices are sometimes called **data tables**, in which **rows correspond to observations** (e.g., customers) and **columns correspond to features** (e.g., purchases).

- Depending on the context, matrices can be conceptualized as a set of column vectors stacked next to each other (e.g., a data table of observations-by-features), as a set of row vectors layered on top of each other (e.g., multisensor data in which each row is a time series from a different channel), or as an ordered collection of individual matrix elements (e.g., an image in which each matrix element encodes pixel intensity value).

- Although we see an image in this shape, computers see and store image in the form of numbers.

- Each of these pixels is denoted as a numerical value, and these numbers are called **Pixel Values**. These pixel values denote the intensity of the pixels. For a grayscale image, we have pixel values ranging from 0 to 255.

- The smaller numbers closer to zero represent the darker shade while the larger numbers closer to 255 represent the lighter or the white shade.



Images are stored in a computer as a matrix of numbers known as pixel values.

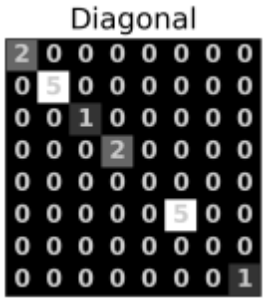
Matrix

➢ Special Matrices

- There is an infinite number of matrices, because there is an infinite number of ways of organizing numbers into a matrix. But matrices can be described using a relatively small number of characteristics, which creates "families" or categories of matrices. These categories are important to know, because they appear in certain operations or have certain useful properties.

- **Random numbers matrix:** This is a matrix that contains numbers drawn at random from some **distribution**, typically **Gaussian (a.k.a. normal)**. Random-numbers matrices are great for exploring linear algebra in code, because they are quickly and easily created with any size and rank.

- **Square versus nonsquare:** A square matrix has the same number of rows as columns; in other words, the matrix is in $\mathbb{R}^{N \times N}$. A nonsquare matrix, also sometimes called a **rectangular** matrix, has a different number of rows and columns. Rectangular matrices are called **tall** if they have more rows than columns and **wide** if they have more columns than rows.

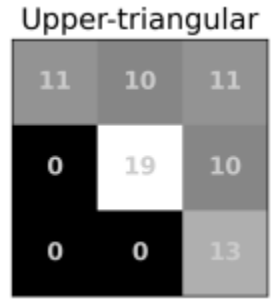


- **Diagonal:** The **diagonal** of a matrix is the elements starting at the top-left and going down to the bottom-right. A diagonal matrix has zeros on all the off-diagonal elements; the diagonal elements may also contain zeros, but they are the only elements that may contain nonzero values.

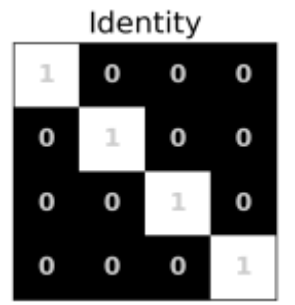
Matrix

➢ Special Matrices

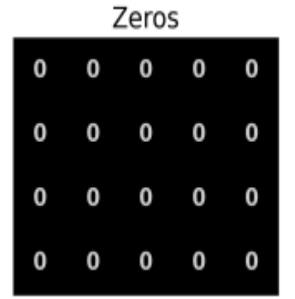
- **Triangular:** A triangular matrix contains all zeros either above or below the main diagonal. The matrix is called **upper triangular** if the nonzero elements are above the diagonal and **lower triangular** if the nonzero elements are below the diagonal.



- **Identity:** The identity matrix is one of the most important special matrices. It is the equivalent of the number 1, in that any matrix or vector times the identity matrix is that same matrix or vector. The identity matrix is a **square diagonal matrix** with all diagonal elements having a value of 1. It is indicated using the letter $\boldsymbol{I}$. You might see a subscript to indicate its size (e.g., $\boldsymbol{I}_5$ is the 5 × 5 identity matrix); if not, then you can infer the size from context (e.g., to make the equation consistent).



- **Zeros:** The zeros matrix is comparable to the zeros vector: it is the matrix of all zeros. Like the zeros vector, it is indicated using a bold-faced zero: **0**. It can be a bit confusing to have the same symbol indicate both a vector and a matrix, but this kind of overloading is common in math and science notation.

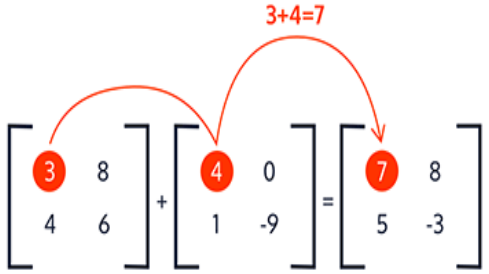


Matrix

➢ Matrix Math: Addition, Scalar Multiplication, Hadamard Multiplication

➢ Addition and Subtraction

- We can add matrices to each other, as long as they have the same shape, just by adding their corresponding elements: $\boldsymbol{C} = \boldsymbol{A} + \boldsymbol{B}$ where $C_{i,j} = A_{i,j} + B_{i,j}$.



➢ Shifting a Matrix

- As with vectors, it is not formally possible to add a scalar to a matrix, as in $\lambda + \boldsymbol{A}$. Python allows such an operation (e.g., 3+np.eye(2)), which involves broadcastadding the scalar to each element of the matrix. That is a convenient computation, but it is not formally a linear algebra operation.

- But there is a linear-algebra way to add a scalar to a square matrix, and that is called **shifting** a matrix. It works by adding a constant value to the diagonal, which is implemented by adding a scalar multiplied identity matrix:

$$\boldsymbol{A} + \lambda \boldsymbol{I}$$

Here's a numerical example:

$$\begin{bmatrix} 4 & 5 & 1 \\ 0 & 1 & 11 \\ 4 & 9 & 7 \end{bmatrix} + 6 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 10 & 5 & 1 \\ 0 & 7 & 11 \\ 4 & 9 & 13 \end{bmatrix}$$

- Notice that only the diagonal elements change; the rest of the matrix is unadulterated by shifting.

- "**Shifting**" a matrix has two primary (extremely important!) applications: it is the mechanism of finding the **eigenvalues** of a matrix, and it is the mechanism of **regularizing** matrices when **fitting models to data**.

Matrix

➢ Matrix Math: Addition, Scalar Multiplication, Hadamard Multiplication

➢ Scalar and Hadamard Multiplications

- These two types of multiplication work the same for matrices as they do for vectors, which is to say, element-wise.

- **Scalar-matrix multiplication** means to multiply each element in the matrix by the same scalar. Here is an example using a matrix comprising letters instead of numbers:

$$\gamma \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} \gamma a & \gamma b \\ \gamma c & \gamma d \end{bmatrix}$$

- Likewise, **Hadamard multiplication** involves multiplying two matrices element-wise (hence the alternative terminology **element-wise multiplication**). Here is an example:

$$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \odot \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 2a & 3b \\ 4c & 5d \end{bmatrix}$$

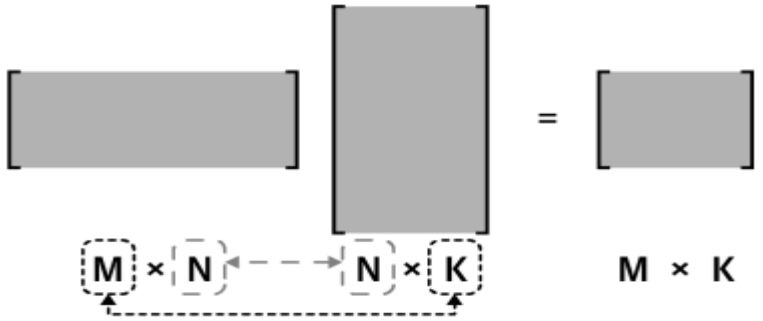
Matrix

➢ Standard Matrix Multiplication

- But before we get into the details of how to multiply two matrices, we will first investigate how to determine whether two matrices can be multiplied.

➢ Rules for Matrix Multiplication Validity

- Here's the important point: matrix multiplication is valid only when the "inner" dimensions match, and the size of the product matrix is defined by the "outer" dimensions.



Matrix multiplication validity, visualized. Memorize this picture.

- You can already see that matrix multiplication does not obey the **commutative law**: $\boldsymbol{AB}$ may be valid while $\boldsymbol{BA}$ is invalid. Even if both multiplications are valid (for example, if both matrices are square), they may produce different results. That is, if $\boldsymbol{C = AB}$ and $\boldsymbol{D = BA}$, then in general $\boldsymbol{C \neq D}$ (they are equal in some special cases, but we cannot generally assume equality).

- Note the notation: Hadamard multiplication is indicated using a dotted-circle ($\boldsymbol{A \odot B}$) whereas matrix multiplication is indicated as two matrices side-by-side without any symbol between them ($\boldsymbol{AB}$).

Matrix

➢ Standard Matrix Multiplication

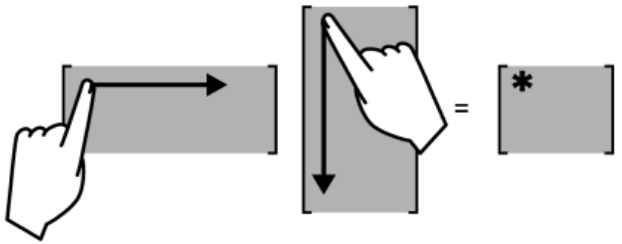
- Now it's time to learn about the mechanics and interpretation of matrix multiplication.

➢ Matrix Multiplication

- Example of matrix multiplication. Parentheses added to facilitate visual grouping.

- If you are struggling to remember how matrix multiplication works, the figure on the right shows a mnemonic trick for drawing out the multiplication with your fingers.

$$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} (2a+3c) & (2b+3d) \\ (4a+5c) & (4b+5d) \end{bmatrix}$$



Finger movements for matrix multiplication

- **Question:** How do you **interpret** matrix multiplication? Remember that the dot product is a number that encodes the relationship between two vectors. So, the result of matrix multiplication is a matrix that stores all the pairwise linear relationships between rows of the left matrix and columns of the right matrix. That is a beautiful thing, and is the basis for computing **covariance** and **correlation matrices**, the **general linear model**, **singular-value decomposition**, and countless other applications.

Matrix

➢ Matrix-Vector Multiplication

- In a purely mechanical sense, **matrix-vector multiplication** is nothing special and does not deserve its own subsection: multiplying a matrix and a vector is simply matrix multiplication where one "matrix" is a vector.

- But matrix-vector multiplication does have many applications in data science, machine learning, and computer graphics, so it's worth spending some time on. Let's start with the basics:

  - A matrix can be right-multiplied by a column vector but not a row vector, and it can be left-multiplied by a row vector but not a column vector. In other words, $\boldsymbol{A}\,\boldsymbol{v}$ and $\boldsymbol{v}^T\boldsymbol{A}$ are valid, but $\boldsymbol{A}\,\boldsymbol{v}^T$ and $\boldsymbol{v}\,\boldsymbol{A}$ are invalid. That is clear from inspecting matrix sizes: an M × N matrix can be premultiplied by a **1 × M matrix (a.k.a. a row vector)** or postmultiplied by an **N × 1 matrix (a.k.a. a column vector).**

  - The result of matrix-vector multiplication is always a vector, and the orientation of that vector depends on the orientation of the multiplicand vector: premultiplying a matrix by a row vector produces another row vector, while postmultiplying a matrix by a column vector produces another column vector. Again, this is obvious when you think about matrix sizes, but it's worth pointing out.

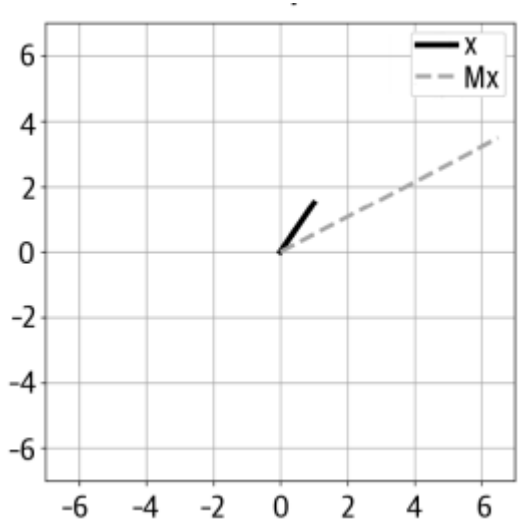
- There are so many more examples of how matrix-vector multiplication is used in **applied linear algebra**, and you will see several of these examples later in this course.

Matrix

➢ Matrix-Vector Multiplication

➢ Geometric transforms

- When we think of a vector as a geometric line, then matrix-vector multiplication becomes a way of **rotating and scaling that vector** (remember that scalar-vector multiplication can scale but not rotate).
- Let's start with a 2D case for easy visualization.
- The figure below visualizes two vectors. You can see that the matrix ***M*** both **rotated** and **stretched** the original vector.



Example of matrix-vector multiplication

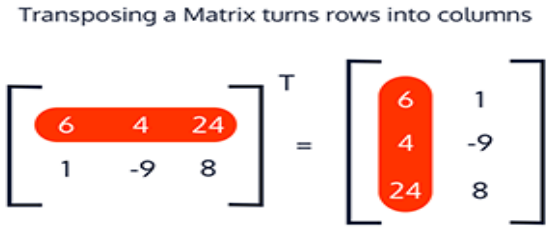
Matrix

➢ Matrix Operations: Transpose

- You learned about the **transpose** operation on vectors. The principle is the same with matrices: swap the rows and columns.
- Just like with vectors, the transpose is indicated with a superscripted T (thus, $\boldsymbol{A}^T$ is the transpose of $\boldsymbol{A}$).
- Double-transposing a matrix returns the original matrix $(\boldsymbol{A}^T)^T = \boldsymbol{A}$.
- The formal **mathematical definition** of the transpose operation is:

$$(\boldsymbol{A}^T)_{i,j} = \boldsymbol{A}_{j,i}$$

- Here's an example:



- $\boldsymbol{A}$, $\boldsymbol{B}$, and $\boldsymbol{C}$ are all matrices, and you can assume that their sizes match to make multiplication valid, then:

$$(\boldsymbol{ABC})^T = \boldsymbol{C}^T\boldsymbol{B}^T\boldsymbol{A}^T$$

Matrix

➢ Symmetric Matrices

- A Symmetric matrix equals its transpose. In math terms, a matrix A is symmetric if $A^T = A$.
- **Example:** note that each row equals its corresponding column

$$\begin{bmatrix} a & e & f & g \\ e & b & h & i \\ f & h & c & j \\ g & i & j & d \end{bmatrix}$$

- **Question:** Can a nonsquare matrix be symmetric? Nope! The reason is that if a matrix is of size M × N , then its transpose is of size N × M . Those two matrices could not be equal unless M = N , which means the matrix is square.

➢ Creating Symmetric Matrices from Nonsymmetric Matrices

- This may be surprising at first, but multiplying ***any*** matrix—**even a nonsquare and nonsymmetric matrix**—by its transpose will produce a **square symmetric matrix**. In other words, $A^T A$ is square symmetric, as is $A A^T$.
- Proof

Summary

- Matrices are array of numbers. In different applications, it is useful to conceptualize them as a set of column vectors, a set of row vectors, or an arrangement of individual values.
- There are several categories of special matrices. Being familiar with the properties of the types of matrices will help you understand matrix equations and advanced applications.
- Some arithmetic operations work element-wise, like addition, scalar multiplication, and Hadamard multiplication.
- Shifting a matrix means adding a constant to the diagonal elements (without changing the off-diagonal elements). Shifting has several applications in machine learning, primarily for finding eigenvalues and regularizing statistical models.
- Matrix multiplication involves dot products between rows of the left matrix and columns of the right matrix. Memorize the rule for matrix multiplication validity: (M × N) (N × K) = (M × K).
- $(\boldsymbol{ABC})^T = \boldsymbol{C}^T\boldsymbol{B}^T\boldsymbol{A}^T$
- A matrix $\boldsymbol{A}$ is symmetric if $\boldsymbol{A}^T = \boldsymbol{A}$. Symmetric matrices have many interesting and useful properties that make them great to work with in applications.
- You can create a symmetric matrix from any matrix by multiplying that matrix by its transpose ($\boldsymbol{A}^T\boldsymbol{A}$ *and* $\boldsymbol{A}\,\boldsymbol{A}^T$).

Matrix

➢ Matrix Norms

- You learned about vector norms: the norm of a vector is its Euclidean geometric length, which is computed as the square root of the sum of the squared vector elements.

- The **Euclidean norm** is also called the **Frobenius norm**, and is computed as the square root of the sum of all matrix elements squared:

$$\boldsymbol{A} = \begin{bmatrix} a_{1,1} & \cdots & a_{1,N} \\ \vdots & \ddots & \vdots \\ a_{M,1} & \cdots & a_{M,N} \end{bmatrix}, \qquad \|\boldsymbol{A}\|_F \triangleq \sqrt{\sum_{i=1}^{M}\sum_{j=1}^{N} a_{i,j}^2}$$

- The Frobenius norm is also called the **ℓ2 norm** (the ℓ is a fancy-looking letter L). And the ℓ2 norm gets its name from the general formula for element-wise ***p*-norms** (notice that you get the Frobenius norm when $p = 2$):

$$\|\boldsymbol{A}\|_p \triangleq \left(\sum_{i=1}^{M}\sum_{j=1}^{N} |a_{i,j}|^p\right)^{1/p}$$

- Matrix norms have several applications in **machine learning** and **statistical analysis**. One of the important applications is in **regularization**, which aims to improve model fitting and increase **generalization** of models to unseen data.

Matrix

➢ Matrix Norms

- The basic idea of regularization is to add a matrix norm as a cost function to a minimization algorithm. That norm will help prevent **model parameters** from becoming too large (ℓ2 regularization, also called **ridge regression**) or encouraging sparse solutions (ℓ1 regularization, also called **lasso regression**).

- Another application of the Frobenius norm is computing a measure of "**matrix distance**." The distance between a matrix and itself is 0, and the distance between two distinct matrices increases as the numerical values in those matrices become increasingly dissimilar. Frobenius matrix distance is computed simply by replacing matrix $\boldsymbol{A}$ with matrix $\boldsymbol{C} = \boldsymbol{A} - \boldsymbol{B}$ in Frobenius norm equation.

- This distance can be used as an **optimization criterion** in machine learning algorithms, for example, to reduce the data storage size of an image while minimizing the Frobenius distance between the reduced and original matrices.

➢ Matrix Trace and Frobenius Norm

- The **trace** of a matrix is the sum of its diagonal elements, indicated as tr($\boldsymbol{A}$) , and exists only for square matrices. Both of the following matrices have the same trace (14):

$$\begin{bmatrix} 4 & 5 & 6 \\ 0 & 1 & 4 \\ 9 & 9 & 9 \end{bmatrix}, \quad \begin{bmatrix} 0 & 0 & 0 \\ 0 & 8 & 0 \\ 1 & 2 & 6 \end{bmatrix}$$

Matrix

➢ Matrix Trace and Frobenius Norm

- Many properties of the trace are less relevant for data science applications, but here is one interesting exception:

$$\|\boldsymbol{A}\|_F \triangleq \sqrt{\sum_{i=1}^{M}\sum_{j=1}^{N} a_{i,j}^2} = \sqrt{tr(\boldsymbol{A}^T\boldsymbol{A})}$$

- In other words, the Frobenius norm can be calculated as the square root of the trace of the matrix times its transpose. The reason why this works is that each diagonal element of the matrix $\boldsymbol{A}^T\boldsymbol{A}$ is defined by the dot product of each row with itself.

➢ Matrix Spaces (Column, Row, Nulls)

- The concept of **matrix spaces** is central to many topics in abstract and applied linear algebra. Fortunately, matrix spaces are conceptually straightforward, and are essentially just linear weighted combinations of different features of a matrix.

➢ Column Space

- Remember that a linear weighted combination of vectors involves scalar multiplying and summing a set of vectors.

Matrix

➢ Column Space

- **Definition**

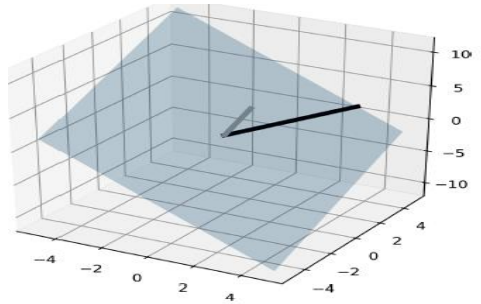
> The **range** of a matrix $\boldsymbol{A} = [\boldsymbol{a}_1 \ldots \boldsymbol{a}_N]$; also known as its **column space**, is the span of its columns:
>
> $$\mathrm{C}(\boldsymbol{A}) \triangleq span(\{\boldsymbol{a}_1, \ldots, \boldsymbol{a}_N\})$$

- First, we conceptualize a matrix as a set of column vectors. Second, we consider the infinity of real-valued scalars instead of working with a specific set of scalars. An infinite number of scalars gives an infinite number of ways to combine a set of vectors. That resulting infinite set of vectors is called the **column space of a matrix**.

- **Question:** What is the dimensionality of column space? The dimensionality of the column space equals the number of columns only if the columns form a linearly independent set. (Remember that linear independence means a set of vectors in which no vector can be expressed as a linear weighted combination of other vectors in that set.)

- **Example**

There are two columns in $\mathbb{R}^3$. Those two columns are linearly independent, meaning you cannot express one as a scaled version of the other. So the column space of this matrix is 2D, and it's a 2D plane that is embedded in $\mathbb{R}^3$.



The 2D column space of a matrix embedded in 3D. The two thick lines depict the two columns of the matrix.

Matrix

➢ Row Space

- Once you understand the column space of a matrix, the **row space** is really easy to understand. In fact, the row space of a matrix is the exact same concept but we consider all possible weighted combinations of the rows instead of the columns.

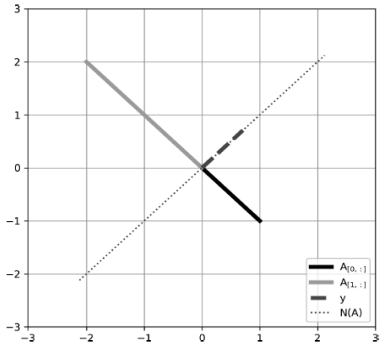
- Row space is indicated as R($\boldsymbol{A}$) . And because the transpose operation swaps rows and columns, you can also write that the row space of a matrix is the column space of the matrix transposed, in other words,

$$\mathrm{R}(\boldsymbol{A}) = \mathrm{C}(\boldsymbol{A}^T) .$$

➢ Null space or kernel

- **Definition**

> The **null space** or **kernel** of a M × N matrix $\boldsymbol{A}$ is:
> $$\mathrm{N}(\boldsymbol{A}) \triangleq \{\boldsymbol{y} \in \mathbb{R}^n : \boldsymbol{A}\,\boldsymbol{y} = \boldsymbol{0}\}$$



Visualization of the null space of a matrix

- N($\boldsymbol{A}$) is indeed a **subspace**.

- But they key point is this: the null space is empty when the columns of the matrix form a linearly independent set.

Matrix

➢ Rank of a matrix

- **Definition**

For any M × N matrix $\boldsymbol{A}$:

column rank of $\boldsymbol{A}$ ≜ dim(C($\boldsymbol{A}$)) = number of linearly independent columns of $\boldsymbol{A}$ ≤ N

row rank of $\boldsymbol{A}$ ≜ dim(C($\boldsymbol{A}^T$)) = number of linearly independent rows of $\boldsymbol{A}$ ≤ M

- **Theorem:** For any M × N matrix $\boldsymbol{A}$: column rank of $\boldsymbol{A}$ = row rank of $\boldsymbol{A}$
- **Corollary**: $rank(\boldsymbol{A}) \leq min(M, N)$
- **Example:** try to guess the rank of each matrix based on the definition

$$\mathbf{A} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 3 \\ 2 & 6 \\ 4 & 12 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 3.1 \\ 2 & 6 \\ 4 & 12 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 1 & 3 & 2 \\ 6 & 6 & 1 \\ 4 & 2 & 0 \end{bmatrix}, \quad \mathbf{E} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{F} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

➢ Rank of Added and Multiplied Matrices

$$rank(\boldsymbol{A} + \boldsymbol{B}) \leq rank(\boldsymbol{A}) + rank(\boldsymbol{B})$$

$$rank(\boldsymbol{AB}) \leq min\{rank(\boldsymbol{A}), rank(\boldsymbol{B})\}$$

Matrix

➢ One of the Applications of Rank: Linear Independence of a Vector Set

- Now that you know about matrix rank, you are ready to understand an algorithm for determining whether a set of vectors is linearly independent.

- The algorithm is straightforward:

    1. Form a matrix with the vectors as columns(or rows).
    2. Compute the rank of the matrix.
    3. If the rank equals the number of vectors, the vectors are linearly independent; otherwise, they are linearly dependent.

Matrix

➢ Matrix Inverse

- Solving **matrix equations** is a huge part of data science.
- The **matrix inverse** is central to solving matrix equations in practical applications, including fitting statistical models to data.
- For a nonzero scalar x, we know that $x\frac{1}{x} = 1$. Matrix inversion is the **generalization** of this identity.
- The inverse of matrix $\boldsymbol{A}$ is another matrix $\boldsymbol{A^{-1}}$(pronounced "A inverse") that multiplies $\boldsymbol{A}$ to produce the **identity matrix.** In other words, $\boldsymbol{A^{-1}A = I}$.
- **Question**: But why do we even need to invert matrices? We need to "cancel" a matrix in order to solve problems that can be expressed in the form $\boldsymbol{Ax = b}$, where $\boldsymbol{A}$ and $\boldsymbol{b}$ are known quantities and we want to solve for $\boldsymbol{x}$. The solution has the following general form:

$$\begin{aligned} \mathbf{Ax} &= \mathbf{b} \\ \mathbf{A^{-1}Ax} &= \mathbf{A^{-1}b} \\ \mathbf{Ix} &= \mathbf{A^{-1}b} \\ \mathbf{x} &= \mathbf{A^{-1}b} \end{aligned}$$

- Unfortunately, life is not always so simple: **not all matrices can be inverted**

Matrix

➢ Matrix Inverse

- There are three different kinds of inverses that have different conditions for **invertibility**. They are introduced here:

  - **Full inverse**

    This means $\boldsymbol{A^{-1}A = A\,A^{-1} = I}$. There are two conditions for a matrix to have a full inverse: (1) **square** and (2) **full-rank**. Every square full-rank matrix has an inverse, and every matrix that has a full inverse is square and full-rank.

  - **One-sided inverse**
    - A one-sided inverse can transform a rectangular matrix into the identity matrix, but it works only for one multiplication order. In particular, a tall matrix $\boldsymbol{T}$ can have *a* **left-inverse**, meaning $\boldsymbol{LT = I}$ but $\boldsymbol{TL \neq I}$. And a wide matrix **W** can have a **right-inverse**, meaning that $\boldsymbol{WR = I}$ but $\boldsymbol{RW \neq I}$.
    - A nonsquare matrix has a one-sided inverse only if it has the maximum possible rank.

  - **Pseudoinverse**
    - Every matrix has a **pseudoinverse**, regardless of its shape and rank.
    - If the matrix is square full-rank, then its pseudoinverse equals its full inverse.
    - Likewise, if the matrix is nonsquare and has its maximum possible rank, then the pseudoinverse equals its left-inverse (for a tall matrix) or its right-inverse (for a wide matrix).
    - But a **reduced-rank matrix** still has a pseudoinverse, in which case the pseudoinverse transforms the singular matrix into another matrix that is close but not equal to the identity matrix.

Matrix

➢ Matrix Inverse

- Matrices that do not have a full or one-sided inverse are called **singular** or **noninvertible**. That is the same thing as labeling a matrix **reduced-rank** or **rank-deficient**.

➢ Computing the Inverse

- There is an algorithm for computing the inverse of any invertible matrix. It's long and tedious (this is why we have computers do the number crunching for us!), but there are a few shortcuts for special matrices.

- **Inverse of a 2×2 Matrix**

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$\mathbf{A}^{-1} = \frac{1}{ad - bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$\mathbf{A}\mathbf{A}^{-1} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}\frac{1}{ad - bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$= \frac{1}{ad - bc}\begin{bmatrix} ad - bc & 0 \\ 0 & ad - bc \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- **Inverse of a Diagonal Matrix**

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 4 \end{bmatrix}\begin{bmatrix} b & 0 & 0 \\ 0 & c & 0 \\ 0 & 0 & d \end{bmatrix} = \begin{bmatrix} 2b & 0 & 0 \\ 0 & 3c & 0 \\ 0 & 0 & 4d \end{bmatrix}$$

  - **Question**: What happens when you have a diagonal matrix with a zero on the diagonal? You cannot invert that element because you'll have 1/0. So a diagonal matrix with at least one zero on the diagonal has no inverse.
  - Also remember that a diagonal matrix is full-rank only if all diagonal elements are nonzero.

Matrix

➢ Orthogonality

- **Definition**

> We say two vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ (of same length) are **orthogonal** (or **perpendicular**) if their inner product is zero: $\boldsymbol{x}^T\boldsymbol{y} = 0$. In such cases we write $\boldsymbol{x} \perp \boldsymbol{y}$.
>
> If, in addition, $\boldsymbol{x}^T\boldsymbol{x} = \boldsymbol{y}^T\boldsymbol{y} = 1$ (*i.e.*, both **unit norm**), then we call them **orthonormal** vectors.

- **Definition**

> An **orthogonal matrix** is a square matrix whose rows are mutually orthonormal and whose columns are mutually orthonormal. We say a (square) matrix $\boldsymbol{A} \in \mathbb{R}^{N\times N}$ is an orthogonal matrix iff $\boldsymbol{A}^T\boldsymbol{A} = \boldsymbol{A}\,\boldsymbol{A}^T = \boldsymbol{I}$.

- This implies that

$$\boldsymbol{A}^{-1} = \boldsymbol{A}^T$$

- So orthogonal matrices are of interest because their inverse is very cheap to compute.

Matrix

➢ Left Inverse

- A **tall matrix** does not have a full inverse. That is, for matrix T of size $M > N$, there is no tall matrix $\boldsymbol{T^{-1}}$ such that $\boldsymbol{T\,T^{-1} = T^{-1}T = I}$.

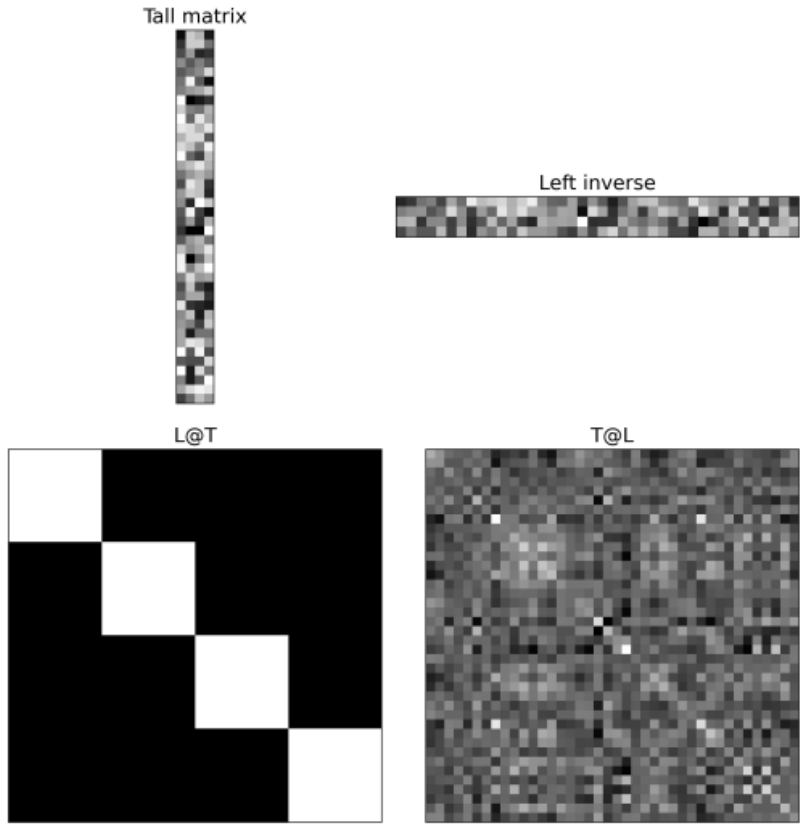
- But there is a matrix $\boldsymbol{L}$ such that $\boldsymbol{LT = I}$.

$$\boldsymbol{L = (T^{\mathrm{T}}T)^{-1}T^{\mathrm{T}}}$$

- The left-inverse is defined only for tall matrices that have full column-rank.

- Figure illustrates a tall matrix, its left-inverse, and the two ways of multiplying the left-inverse by the matrix. Note that the left-inverse is not a square matrix and that postmultiplying by the left-inverse gives a result that is definitely not the identity matrix.

- The left-inverse is extremely important. In fact, once you learn about fitting statistical models to data and the least squares solution, you'll see the left-inverse all over the place. It is no hyperbole to state that the left-inverse is one of the most important contributions of linear algebra to modern human civilization.



Matrix

➢ The Inverse is Unique

- **The matrix inverse is unique,** meaning that if a matrix has an inverse, it has exactly one inverse. There cannot be two matrices $\boldsymbol{B}$ and $\boldsymbol{C}$ such that $\boldsymbol{AB} = \boldsymbol{I}$ and $\boldsymbol{AC} = \boldsymbol{I}$ while $\boldsymbol{B} \neq \boldsymbol{C}$.

➢ Moore-Penrose Pseudoinverse

- **Reduced-rank** matrices do not have a **full** or a **one-sided inverse**.

- But these matrices do have **pseudoinverses**. Pseudoinverses are transformation matrices that bring a matrix close to the identity matrix.

- The plural ***pseudoinverses*** was not a typo: although the full matrix inverse is unique, the pseudoinverse is not unique. A reduced-rank matrix has an infinite number of pseudoinverses. But some are better than others, and there is really only one pseudoinverse worth discussing, because it is most likely the only one you'll ever use.

- That is called the **Moore-Penrose pseudoinverse**, sometimes abbreviated as the MP pseudoinverse. But because this is by far the most commonly used pseudoinverse, you can always assume that the term **pseudoinverse** refers to the Moore-Penrose pseudoinverse.
- The pseudoinverse is indicated using a dagger, a plus sign, or an asterisk in the superscript: $\boldsymbol{A}^{\dagger}$, $\boldsymbol{A}^{+}$, or $\boldsymbol{A}^{*}$.
- The pseudoinverse is implemented in Python using the function np.linalg.pinv().

Summary

- Here is a list of the most important points:
  - The most commonly used norm is called the **Frobenius norm** (a.k.a. the **Euclidean norm** or the **ℓ2 norm**) and is computed as the square root of the sum of squared elements.
  - The trace of a matrix is the sum of the diagonal elements.
  - Three matrix spaces (column, row, and null), and they are defined as the set of linear weighted combinations of different features of the matrix.
  - The column space of the matrix comprises all linear weighted combinations of the columns in the matrix and is written as C($\boldsymbol{A}$).
  - The row space of a matrix is the set of linear weighted combintions of the rows of the matrix and is indicated as R($\boldsymbol{A}$) or C($\boldsymbol{A}^T$).
  - The null space of a matrix is the set of vectors that linearly combines the columns to produce the zeros vector—in other words, any vector $\boldsymbol{y}$ that solves the equation $\boldsymbol{A}\,\boldsymbol{y} = \boldsymbol{0}$. The trivial solution of $\boldsymbol{y} = \boldsymbol{0}$ is excluded. The null space is important for finding eigenvectors, among other applications.
  - Rank is a nonnegative integer associated with a matrix. Rank reflects the largest number of columns (or rows) that can form a linearly independent set. Matrices with a rank smaller than the maximum possible are called **reduced-rank** or **singular**.

Summary

- The determinant is a number associated with a square matrix (there is no determinant for rectangular matrices). It is tedious to compute, but the most important thing to know about the determinant is that it is zero for all reduced-rank matrices and nonzero for all full-rank matrices.

- The matrix inverse is a matrix that, through matrix multiplication, transforms a maximum-rank matrix into the identity matrix. The inverse has many purposes, including moving matrices around in an equation (e.g., solve for $\boldsymbol{x}$ in $\boldsymbol{Ax} = \boldsymbol{b}$).

- A **square full-rank** matrix has a **full inverse**, a **tall full column-rank** matrix has a **left-inverse**, and a **wide full row-rank** matrix has a **right-inverse**.

- Orthogonal vectors and matrices

- **Reduced-rank matrices** cannot be linearly transformed into the identity matrix, but they do have a **pseudoinverse** that transforms the matrix into another matrix that is closer to the identity matrix.

- The inverse is unique—if a matrix can be linearly transformed into the identity matrix, there is only one way to do it.

- There are some tricks for computing the inverses of some kinds of matrices, including 2×2 and diagonal matrices. These shortcuts are simplifications of the full formula for computing a matrix inverse.

Matrix Decompositions

- To "**decompose**" a vector or matrix means to break up that matrix into multiple simpler pieces. Decompositions are used to reveal information that is "hidden" in a matrix, to make the matrix easier to work with, or for data compression.

- It is no understatement to write that much of linear algebra (in the abstract and in practice) involves matrix decompositions. **Matrix decompositions are a big deal.**

- introducing the concept of a decomposition using two simple examples with **scalars**:

  - We can decompose the number 42.01 into two pieces: 42 and 0.01. Perhaps 0.01 is noise to be ignored, or perhaps the goal is to compress the data (the integer 42 requires less memory than the floating-point 42.01). Regardless of the motivation, the decomposition involves representing one mathematical object as the sum of simpler objects (42.01 = 42 + 0.01).

  - We can decompose the number 42 into the product of **prime numbers** 2, 3, and 7. This decomposition is called **prime factorization** and has many applications in numerical processing and cryptography. This example involves products instead of sums, but the point is the same: decompose one mathematical object into smaller, simpler pieces.

- Much as we can discover something about the true nature of an integer by decomposing it into prime factors, we can also decompose matrices in ways that show us information about their functional properties that is not obvious from the representation of the matrix as an array of elements.

Matrix Decompositions

- There are many matrix decompositions in linear algebra. We focus on the two particularly important matrix decompositions: the **eigendecomposition** and the **singular value decomposition(SVD)**

- Eigendecomposition and SVD, are among the most important contributions of linear algebra to data science and machine learning.

➢ **Eigendecomposition**

- **The same object can be seen in different ways depending on its use.** So it is with **eigendecomposition**: eigendecomposition has a geometric interpretation (axes of rotational invariance), a statistical interpretation (directions of maximal covariance), a dynamical-systems interpretation (stable system states), a graph-theoretic interpretation (the impact of a node on its network), a financial-market interpretation (identifying stocks that covary), and many more.

- Eigendecomposition is defined only for square matrices. It is not possible to eigendecompose an M × N matrix unless M = N.

- Nonsquare matrices can be decomposed using the SVD.

- Every square matrix of size M × M has *M* eigenvalues (scalars) and *M* corresponding eigenvectors. The purpose of eigendecomposition is to reveal those *M* vector-scalar pairs.

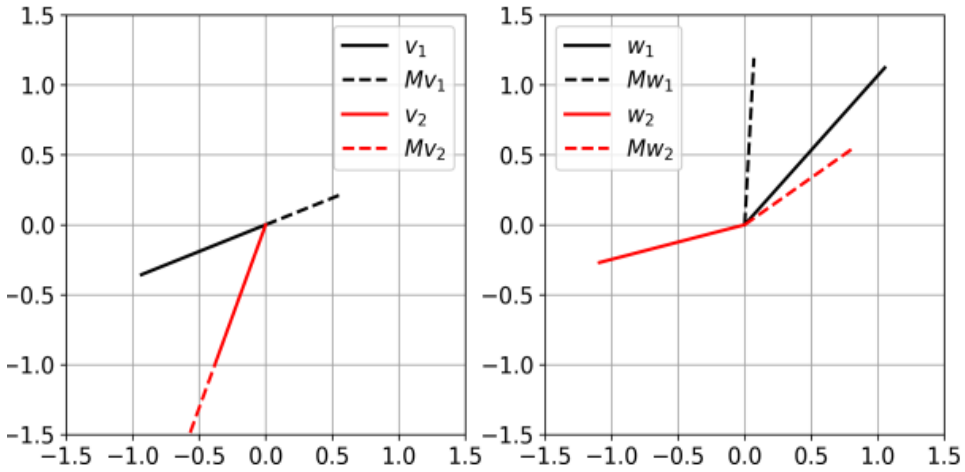
Matrix Decompositions

➢ Interpretations of Eigenvalues and Eigenvectors

- There are several ways of **interpreting** eigenvalues/vectors. Of course, the math is the same regardless.

➢ Geometric Interpretation

- Figure shows vectors before and after multiplication by a 2×2 matrix. The two vectors in the left plot ($\boldsymbol{v}_1$ and $\boldsymbol{v}_2$) are eigenvectors whereas the two vectors in the right plot are not. The eigenvectors point in the same direction before and after postmultiplying the matrix. The eigenvalues encode the amount of stretching; try to guess the eigenvalues based on visual inspection of the plot.



Geometry of eigenvectors

- That's the geometric picture: an eigenvector means that matrix-vector multiplication acts like scalar-vector multiplication. Let's see if we can write that down in an equation

$$\boldsymbol{A}\,\boldsymbol{v} = \lambda\,\boldsymbol{v}$$

- This is called the **eigenvalue equation**, and it's another key formula in linear algebra that is worth memorizing.

Matrix Decompositions

➢ Geometric Interpretation

- Be careful with interpreting that equation: it does not say that the matrix *equals* the scalar; it says that the *effect* of the matrix on the vector is the same as the *effect* of the scalar on that same vector.

➢ Finding Eigenvalues and Eigenvectors

- To eigendecompose a square matrix, you first find the eigenvalues, and then use each eigenvalue to find its corresponding eigenvector.

▪ Approach 1: by hand

- Eigenvalue equation:

$$\begin{aligned} \mathbf{Av} &= \lambda\mathbf{v} \\ \mathbf{Av} - \lambda\mathbf{v} &= \mathbf{0} \\ (\mathbf{A} - \lambda\mathbf{I})\mathbf{v} &= \mathbf{0} \end{aligned}$$

$$|\mathbf{A} - \lambda\mathbf{I}| = 0$$

- Traditional linear algebra textbook
- Tedious Practice Problems

▪ Approach 2: by code

- Finding eigenvalues and eigenvectors is easy in Python:

```
matrix = np.array([
                   [1,2],
                   [3,4]
                   ])

evals,evecs = np.linalg.eig(matrix)
print(evals), print(evecs)
```

```
[-0.37228132  5.37228132]

[[-0.82456484 -0.41597356]
 [ 0.56576746 -0.90937671]]
```

Matrix Decompositions

➢ Sign and Scale Indeterminacy of Eigenvectors

- In other words, if $\boldsymbol{v}$ is an eigenvector of a matrix, then so is $\alpha\boldsymbol{v}$ for any real-valued α except zero.
- Indeed, eigenvectors are important because of their **direction**, not because of their **magnitude.**
- The infinity of possible null space basis vectors leads to two questions:
  - **Question:** *Is there one "best" basis vector?* There isn't a "best" basis vector per se, but it is convenient to have eigenvectors that are **unit normalized (a Euclidean norm of 1)**. This is particularly useful for symmetric matrices.
  - **Question:** *What is the "correct" sign of an eigenvector?* There is none. In fact, you can get different eigenvector signs from the same matrix when using different versions of NumPy—as well as different software such as MATLAB, or Julia. Eigenvector sign indeterminacy is just a feature of life in our universe. In applications such as PCA, there are principled ways for assigning a sign, but that's just common convention to facilitate interpretation.

Matrix Decompositions

➢ Diagonalizing a Square Matrix

- The eigenvalue equation that you are now familiar with lists one eigenvalue and one eigenvector. This means that an M × M matrix has *M* eigenvalue equations:

$$\begin{aligned}\mathbf{A}\mathbf{v}_1 &= \lambda_1\mathbf{v}_1\\ &\vdots\\ \mathbf{A}\mathbf{v}_M &= \lambda_M\mathbf{v}_M\end{aligned}$$

- There's nothing really wrong with that series of equations, but it is kind of ugly, and ugliness violates one of the principles of linear algebra: make equations compact and elegant. Therefore, we transform this series of equations into one **matrix equation.**

- The following equation shows the form of diagonalization for a 3 × 3 matrix (using @ in place of numerical values in the matrix). In the eigenvectors matrix, the first subscript number corresponds to the eigenvector, and the second subscript number corresponds to the eigenvector element. For example, v_{12} is the second element of the first eigenvector:

$$\begin{bmatrix}@ & @ & @\\ @ & @ & @\\ @ & @ & @\end{bmatrix}\begin{bmatrix}v_{11} & v_{21} & v_{31}\\ v_{12} & v_{22} & v_{32}\\ v_{13} & v_{23} & v_{33}\end{bmatrix} = \begin{bmatrix}v_{11} & v_{21} & v_{31}\\ v_{12} & v_{22} & v_{32}\\ v_{13} & v_{23} & v_{33}\end{bmatrix}\begin{bmatrix}\lambda_1 & 0 & 0\\ 0 & \lambda_2 & 0\\ 0 & 0 & \lambda_3\end{bmatrix}$$
$$= \begin{bmatrix}\lambda_1 v_{11} & \lambda_2 v_{21} & \lambda_3 v_{31}\\ \lambda_1 v_{12} & \lambda_2 v_{22} & \lambda_3 v_{32}\\ \lambda_1 v_{13} & \lambda_2 v_{23} & \lambda_3 v_{33}\end{bmatrix}$$

Matrix Decompositions

➢ Diagonalizing a Square Matrix

- More generally, the matrix eigenvalue equation—a.k.a. the **diagonalization of a square matrix**—is:

$$\boldsymbol{A\,V = V\,\Lambda}$$

- By the way, it's often interesting and insightful in mathematics to rearrange equations by solving for different variables. Consider the following list of equivalent declarations:

$$\boldsymbol{A\,V = V\,\Lambda}$$

$$\boldsymbol{A = V\,\Lambda\,V^{-1}}$$

$$\boldsymbol{\Lambda = V^{-1}\,A\,V}$$

- $\boldsymbol{\Lambda}$ is the capital of λ.

- $\boldsymbol{V}$ is (linearly independent) eigenvectors; $\boldsymbol{\Lambda}$ is diagonal matrix of eigenvalues.

Matrix Decompositions

➢ The Special Awesomeness of Symmetric Matrices

- **Symmetric matrices** have special properties that make them great to work with. Now you are ready to learn two more special properties that relate to eigendecomposition.

1. Orthogonal Eigenvectors

- Symmetric matrices have orthogonal eigenvectors. That means that all eigenvectors of a symmetric matrix are pair-wise orthogonal.
- The orthogonal eigenvector property means that the dot product between any pair of eigenvectors is zero, while the dot product of an eigenvector with itself is nonzero.
- This can be written as $\boldsymbol{V^T V = D}$, where $\boldsymbol{D}$ is a **diagonal matrix** with the diagonals containing the norms of the eigenvectors.
- But we can do even better than just a diagonal matrix: recall that eigenvectors are important because of their direction, not because of their magnitude. So an eigenvector can have any magnitude we want(except, obviously, for a magnitude of zero).
- **Question:** Let's scale all eigenvectors so they have unit length. Question for you: if all eigenvectors are orthogonal and have unit length, what happens when we multiply the eigenvectors matrix by its transpose?

Matrix Decompositions

➢ The Special Awesomeness of Symmetric Matrices

1. Orthogonal Eigenvectors

$$\boldsymbol{V}^T\boldsymbol{V} = \boldsymbol{I}$$

- In other words, the eigenvectors matrix of a symmetric matrix is an **orthogonal matrix**! This has multiple implications for data science, including that the eigenvectors are super easy to invert (because you simply transpose them).

2. Real-Valued Eigenvalues

- A second special property of symmetric matrices is that they have **real-valued eigenvalues** (and therefore real-valued eigenvectors).

- Guaranteed real-valued eigenvalues from symmetric matrices is fortunate because **complex numbers** are often confusing to work with. Lots of matrices used in data science are symmetric, and so if you see complex eigenvalues in your data science applications, it's possible that there is a problem with the code or with the data.

- **Orthogonal eigendecomposition (for any real symmetric matrix)**

$$\boldsymbol{A} = \boldsymbol{Q}\,\boldsymbol{\Lambda}\,\boldsymbol{Q}^T$$

- where $\boldsymbol{Q}$ is an **orthogonal matrix** composed of **eigenvectors** of $\boldsymbol{A}$, and $\boldsymbol{\Lambda}$ is a diagonal matrix. The eigenvalue $\Lambda_{i,i}$ is associated with the eigenvector in column i of $\boldsymbol{Q}$, denoted as $\boldsymbol{Q}_{:,i}$.

Matrix Decompositions

- **Gateway to advanced linear algebra**
  - **The Quadratic Form of a Matrix**
    - Consider the following expression:

$$\boldsymbol{w}^{\boldsymbol{T}}\boldsymbol{A}\,\boldsymbol{w} = \alpha$$

    - In other words, we pre- and postmultiply a square matrix by the same vector wand get a scalar. (Notice that this multiplication is valid only for square matrices.)
    - This is called the **quadratic form** on matrix $\boldsymbol{A}$.
  - **Definiteness**
    - **Definiteness** is a characteristic of a **square matrix** and is defined by the signs of the eigenvalues of the matrix, which is the same thing as the signs of the quadratic form results.
    - A matrix whose eigenvalues are all positive is called **positive definite**.
    - A matrix whose eigenvalues are all positive or zero-valued is called **positive semidefinite**(PSD).

Matrix Decompositions

- **Gateway to advanced linear algebra**
  - **Definiteness**
    - Likewise, if all eigenvalues are negative, the matrix is **negative definite**.
    - If all eigenvalues are negative or zero-valued, it is **negative semidefinite**.
    - Positive semidefinite matrices are interesting because they guarantee that $\forall \boldsymbol{x},\ \boldsymbol{x}^T \boldsymbol{A}\ \boldsymbol{x} \geq 0$.
  - **Generalized Eigendecomposition**
    - Consider that the following equation is the same as the fundamental eigenvalue equation:

$$\boldsymbol{A}\ \boldsymbol{v} = \lambda\ \boldsymbol{I}\ \boldsymbol{v}$$

    - This is obvious because $\boldsymbol{I}\ \boldsymbol{v} = \boldsymbol{v}$. **Generalized eigendecomposition** involves replacing the identity matrix with another matrix (not the identity or zeros matrix):

$$\boldsymbol{A}\ \boldsymbol{v} = \lambda\ \boldsymbol{B}\ \boldsymbol{v}$$

    - **NumPy** does not compute generalized eigendecomposition, but **SciPy** does.

Summary

- Here is a reminder of the key points:

  - Eigendecomposition identifies *M* scalar/vector pairs of an M × M matrix. Those pairs of eigenvalue/eigenvector reflect **special directions** in the matrix and have myriad applications in data science.

  - **Diagonalizing a matrix** means to represent the matrix as $\boldsymbol{A = V \Lambda V^{-1}}$, where $\boldsymbol{V}$ is a matrix with eigenvectors in the columns and $\boldsymbol{\Lambda}$ is a diagonal matrix with eigenvalues in the diagonal elements.

  - Symmetric matrices have several special properties in eigendecomposition; the most relevant for data science is that all eigenvectors are pair-wise orthogonal. This means that the matrix of eigenvectors is an orthogonal matrix (when the eigenvectors are unit normalized), which in turn means that the inverse of the eigenvectors matrix is its transpose.

  - The **definiteness** of a matrix refers to the signs of its eigenvalues. In data science, the most relevant categories are **positive (semi)definite**, which means that all eigenvalues are either nonnegative or positive.

Matrix Decompositions

➢ Singular Value Decomposition

- We saw how to **decompose a matrix** into eigenvectors and eigenvalues. The **singular value decomposition (SVD)** provides another way to **factorize a matrix**, into **singular vectors** and **singular values**.

- Every real matrix has a singular value decomposition, but the same is not true of the eigenvalue decomposition. For example, if a matrix is not square, the eigendecomposition is not defined, and we must use a singular value decomposition instead.

- Recall that the eigendecomposition involves analyzing a matrix $\boldsymbol{A}$ to discover a matrix $\boldsymbol{V}$ of eigenvectors and a diagonal matrix of eigenvalues $\boldsymbol{\Lambda}$ such that we can rewrite $\boldsymbol{A}$ as

$$\boldsymbol{A} = \boldsymbol{V}\,\boldsymbol{\Lambda}\,\boldsymbol{V}^{-1}$$

- The singular value decomposition is similar, except this time we will write $\boldsymbol{A}$ as a product of three matrices:

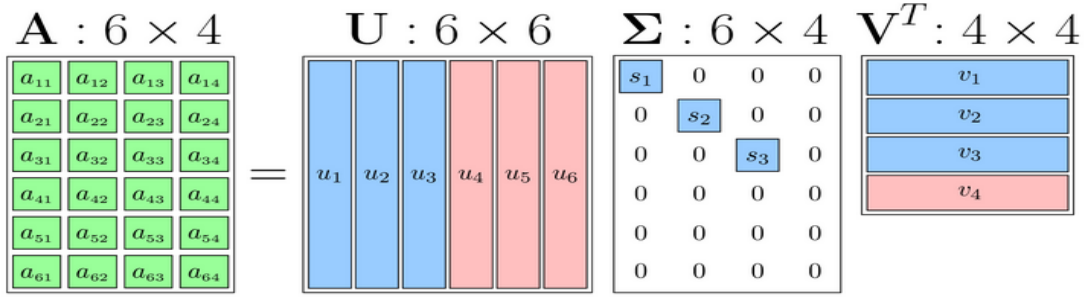
$$\boldsymbol{A} = \boldsymbol{U}\,\boldsymbol{\Sigma}\,\boldsymbol{V}^{\boldsymbol{T}} = \sum_{i=1}^{min\{M,N\}} s_i \boldsymbol{u}_i \boldsymbol{v}_i^T$$

- Suppose that $\boldsymbol{A}$ is an $M \times N$ matrix. Then $\boldsymbol{U}$ is defined to be an $M \times M$ matrix, $\boldsymbol{\Sigma}$ to be an $M \times N$ matrix, and $\boldsymbol{V}$ to be an $N \times N$ matrix.

Matrix Decompositions

➢ Singular Value Decomposition

- $\boldsymbol{U}$ is $M \times M$ and **orthogonal**: $\boldsymbol{U}^T\boldsymbol{U} = \boldsymbol{U}\boldsymbol{U}^T = \boldsymbol{I}$, and its columns consist of M **left singular vectors** of $\boldsymbol{A}$.
- $\boldsymbol{V}$ is $N \times N$ and **orthogonal**: $\boldsymbol{V}^T\boldsymbol{V} = \boldsymbol{V}\boldsymbol{V}^T = \boldsymbol{I}$, and its columns consist of M **right singular vectors** of $\boldsymbol{A}$.
- $\boldsymbol{\Sigma}$ is a $M \times N$ **rectangular diagonal matrix** containing all the $\min\{M, N\}$ **singular values** of $\boldsymbol{A}$.
- The **singular values** $s_1, \ldots, s_{min(M,N)}$ are **real and nonnegative**, even if the matrix contains complex-valued numbers.
- The number of **nonzero singular values** equals the matrix **rank**.
- The singular values matrix is a diagonal matrix of the same size as $\boldsymbol{A}$. **By convention** The singular values are always sorted from largest (top-left) to smallest (lower-right): $s_1 \geq s_2 \geq \cdots \geq s_{\min(M,N)}$.
- **Example:**



Matrix Decompositions

- **SVD and the MP Pseudoinverse(**a generalization of the matrix inverse when the matrix may not be invertible.**)**

  - The SVD of a matrix inverse is quite elegant. Assuming the matrix is square and invertible, we get:

$$\begin{aligned} \mathbf{A}^{-1} &= \left(\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathrm{T}}\right)^{-1} \\ &= \mathbf{V}\boldsymbol{\Sigma}^{-1}\mathbf{U}^{-1} \\ &= \mathbf{V}\boldsymbol{\Sigma}^{-1}\mathbf{U}^{\mathrm{T}} \end{aligned}$$

  - In other words, we only need to invert $\boldsymbol{\Sigma}$, because $\mathbf{U}^{-1} = \mathbf{U}^{\mathrm{T}}$. Furthermore, because $\boldsymbol{\Sigma}$ is a diagonal matrix, its inverse is obtained simply by inverting each diagonal element. On the other hand, this method is still subject to numerical instabilities, because tiny singular values that might reflect precision errors (e.g., 10^{-15}) become significantly large when inverted.

  - The **MP pseudoinverse** is computed almost exactly as the **full inverse** shown in the previous example; the only modification is to invert the **nonzero** diagonal elements in $\boldsymbol{\Sigma}$ instead of trying to to invert all diagonal elements. (In practice, "nonzero" is implemented as above a threshold to account for precision errors.)

  - And that's it! That's how the pseudoinverse is computed. You can see that it's very simple and intuitive, but requires a considerable amount of background knowledge about linear algebra to understand.

  - Even better: because the SVD works on matrices of any size, the MP pseudoinverse can be applied to nonsquare matrices.

Summary

- The SVD is arguably the most important decomposition in linear algebra, because it reveals rich and detailed information about the matrix. Here are the key points:

  - SVD decomposes a matrix (of any size and rank) into the product of three matrices, termed **left singular vectors** $\boldsymbol{U}$, **singular values** $\boldsymbol{\Sigma}$, and **right singular vectors** $\boldsymbol{V}^T$.

  - The number of **nonzero** singular values equals the rank of the matrix.

  - Moore-Penrose pseudoinverse is computed as $\boldsymbol{V}\,\boldsymbol{\Sigma}^{+}\boldsymbol{U}^T$, where $\boldsymbol{\Sigma}^{+}$ is obtained by inverting the nonzero singular values on the diagonal.

Tensor

- **Definition**

> **Tensors:** A **tensor** is just a **container** for data, typically numerical data. It is, therefore, a container for numbers. In some cases we will need an array with **more than two axes**(dimensions). In the general case, an array of numbers arranged on a regular grid with a variable number of axes is known as a tensor. In summary, tensors are a **generalization** of matrices to any number of dimensions.

- We denote a tensor named "A" with this typeface: **A**. We identify the element of **A** at coordinates (i, j, k) by writing $A_{i,j,k}$.

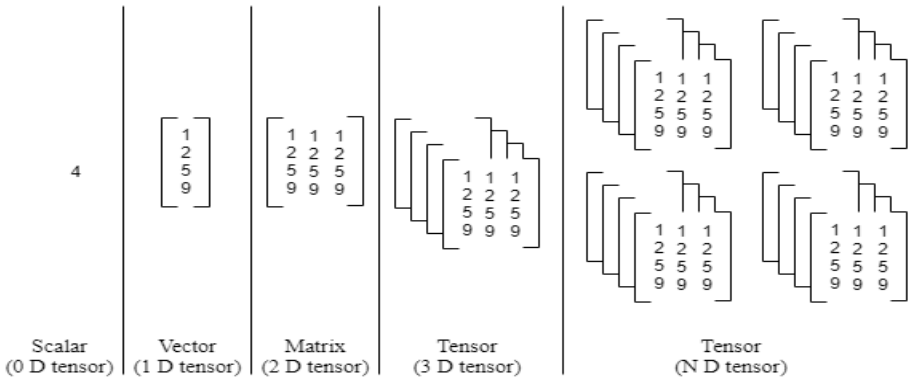
- In the context of machine learning and image processing, A **tensor** can be a generic structure that can be used for **storing**, **representing**, and **changing data**.

- Tensors serve as the **core data structure** for machine and deep learning algorithms. Google's **TensorFlow** got its name because tensors play a crucial role in this field.

- TensorFlow is a free and open-source software library originally developed by Google Brain researchers and developers for machine learning.

TensorFlow

- **Summary**

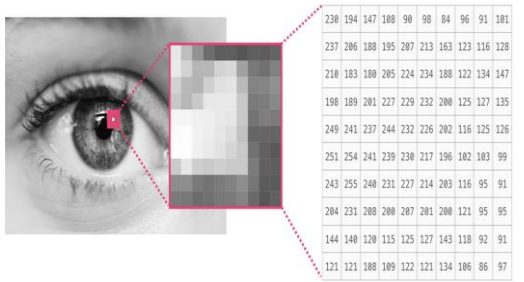


Different types of Tensors

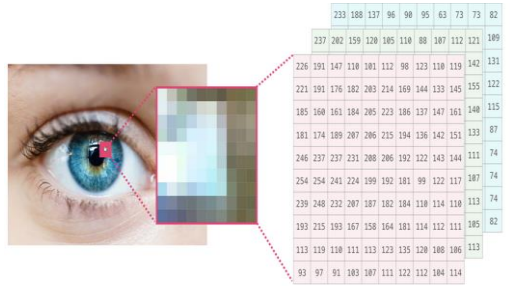
Tensor

➢ Example

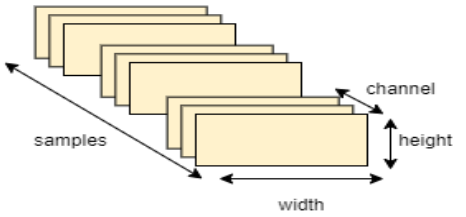
- A grayscale image is a matrix of pixels of size (height × width).
- If the image is **colored**, then each pixel is represented by three numbers - a value for each of red, green, and blue. In that case we need a 3rd dimension (because each cell can only contain one number). So a colored image is represented by an ndarray of dimensions: (height × width × 3).
- An **RGB image** is a practical example of a **tensor**, where the image's height, width, and color channels form a 3-dimensional tensor. This **representation** is crucial for various image processing and machine learning tasks, enabling efficient data manipulation and transformation.
- A **4D tensor** (samples, height, width, channels) can be produced by stacking 3D tensors in an array, and so on. In **deep learning**, you typically work with tensors that range from 0 to 4D, though if you're processing video data, you might go as high as 5D (samples, frames, channels, height, width)



Grayscale Image



Color Image



4D image data tensor

References

[1] Book: “Deep Learning” by Ian Goodfellow
[2] britannica.com/science/linear-algebra
[3] codecademy.com/learn/dsml-math-for-machine-learning/modules/math-ds-linear-algebra/cheatsheet
[4] medium.com/linear-algebra-for-ml-animated
[5] wikipedia.org/wiki/Vector_space
[6] Book: “Practical Linear Algebra for Data Science, From Core Concepts to Applications Using Python” by Mike X Cohen
[7] analyticsvidhya.com/blog/2021/03/grayscale-and-rgb-format-for-storing-images/
[8] truetheta.io/concepts/linear-algebra/svd-and-the-fundamental-theorem-of-linear-algebra
[9] analyticsvidhya.com/blog/2022/07/data-representation-in-neural-networks-tensor/
[10] jalammar.github.io/visual-numpy/
[11] realpython.com/what-can-i-do-with-python/
[12] 3blue1brown.com/topics/linear-algebra